CodeDiff Analyse Report

Generiert am: 5.2.2026, 17:53:35

Zentrale Kennzahlen

Metrik	Wert	Beschreibung
Churn %	17.13%	Intensität der Bearbeitung (Added + Deleted) / Original
Fork Share %	19.67%	Anteil neuen Codes im Zielstand
Net Replacement %	8.56%	Anteil ersetzten Original-Codes

Zusammenfassung Zeilenstatistik

Kategorie	Anzahl Zeilen
Original Gesamt	4.145
Aktuell Gesamt	4.719
Hinzugefügt (mod. Dateien)	355
Gelöscht	355
In neuen Dateien	573

Visualisierung

Detaillierte Dateiliste

Pfad	Status	Add	Del	Gesamt
src/js/helpers/legacy-three-compat.js	ADDED	+491	-0	491
src/js/human/human.js	MODIFIED	+139	-35	642
webpack.config.js	MODIFIED	+41	-77	53
package.json	MODIFIED	+35	-52	58
.eslintrc.js	DELETED	+0	-81	0
webpack.test.config.js	MODIFIED	+31	-36	49
README.md	MODIFIED	+28	-28	28
src/js/human/proxy.js	MODIFIED	+36	-13	273
Installation.md	ADDED	+46	-0	46
test/mocha/fixtures.js	MODIFIED	+19	-11	49
NOTICE.md	ADDED	+21	-0	21
test/mocha/targets_spec.js	MODIFIED	+7	-13	224
COPYRIGHT.md	ADDED	+15	-0	15
src/js/helpers/OBJExporter.js	MODIFIED	+7	-4	159
test/mocha/ethnic-skin-blender_spec.js	MODIFIED	+2	-2	20
src/js/human/human-modifier.js	MODIFIED	+2	-1	794
src/js/human/targets.js	MODIFIED	+2	-1	284
test/mocha/proxy_spec.js	MODIFIED	+2	-1	102
src/js/helpers/helpers.js	MODIFIED	+1	-0	75
src/js/human/ethnic-skin-blender.js	MODIFIED	+1	-0	36
src/js/human/factors.js	MODIFIED	+1	-0	506
src/js/human/index.js	MODIFIED	+1	-0	13

Detaillierte Datei-Gegenüberstellung

FILE: src/js/helpers/legacy-three-compat.js

Status: ADDED | +491 / -0

```
+ import {
+     BufferAttribute,
+     BufferGeometry,
+     Color,
+     FileLoader,
+     Float32BufferAttribute,
+     LoaderUtils,
+     MeshPhongMaterial,
+     TextureLoader,
+     Vector2,
+     Vector3,
+     Vector4
+ } from 'three'
+ import { Face3, Geometry as LegacyGeometry } from 'three-stdlib-geometry'
+
+
+ function isBitSet(value, position) {
+     return value & (1 << position)
+ }
+
+ function parseModel(json, geometry) {
+     let i
+     let j
+     let fi
+     let offset
+     let zLength
+
+     let colorIndex
+     let normalIndex
+     let uvIndex
+     let materialIndex
+
+     let type
+     let isQuad
+     let hasMaterial
+     let hasFaceVertexUv
+     let hasFaceNormal
+     let hasFaceVertexNormal
+     let hasFaceColor
+     let hasFaceVertexColor
+
+     let face
+     let faceA
+     let faceB
+     let hex
+     let normal
+
+     let uvLayer
+     let uv
+     let u
+     let v
+
+     const faces = json.faces || []
+     const vertices = json.vertices || []
+     const normals = json.normals || []
+     const colors = json.colors || []
+     const scale = json.scale || 1
+
+     let nUvLayers = 0
+
+     if (json.uvs !== undefined) {
+         for (i = 0; i < json.uvs.length; i++) {
+             if (json.uvs[i].length) nUvLayers++
```

```
+         }
+
+         for (i = 0; i < nUvLayers; i++) {
+             geometry.faceVertexUvs[i] = []
+         }
+     }
+
+     offset = 0
+     zLength = vertices.length
+
+     while (offset < zLength) {
+         const vertex = new Vector3()
+         vertex.x = vertices[offset++] * scale
+         vertex.y = vertices[offset++] * scale
+         vertex.z = vertices[offset++] * scale
+         geometry.vertices.push(vertex)
+     }
+
+     offset = 0
+     zLength = faces.length
+
+     while (offset < zLength) {
+         type = faces[offset++]
+
+         isQuad = isBitSet(type, 0)
+         hasMaterial = isBitSet(type, 1)
+         hasFaceVertexUv = isBitSet(type, 3)
+         hasFaceNormal = isBitSet(type, 4)
+         hasFaceVertexNormal = isBitSet(type, 5)
+         hasFaceColor = isBitSet(type, 6)
+         hasFaceVertexColor = isBitSet(type, 7)
+
+         if (isQuad) {
+             faceA = new Face3()
+             faceA.a = faces[offset]
+             faceA.b = faces[offset + 1]
+             faceA.c = faces[offset + 3]
+
+             faceB = new Face3()
+             faceB.a = faces[offset + 1]
+             faceB.b = faces[offset + 2]
+             faceB.c = faces[offset + 3]
+
+             offset += 4
+
+             if (hasMaterial) {
+                 materialIndex = faces[offset++]
+                 faceA.materialIndex = materialIndex
+                 faceB.materialIndex = materialIndex
+             }
+
+             fi = geometry.faces.length
+
+             if (hasFaceVertexUv) {
+                 for (i = 0; i < nUvLayers; i++) {
+                     uvLayer = json.uvs[i]
+                     geometry.faceVertexUvs[i][fi] = []
+                     geometry.faceVertexUvs[i][fi + 1] = []
+
+                     for (j = 0; j < 4; j++) {
+                         uvIndex = faces[offset++]
+                         u = uvLayer[uvIndex * 2]
+                         v = uvLayer[uvIndex * 2 + 1]
+                         uv = new Vector2(u, v)
+
+                         if (j !== 2) geometry.faceVertexUvs[i][fi].push(uv)
+                         if (j !== 0) geometry.faceVertexUvs[i][fi + 1].push(uv)
+                     }
+                 }
```

```
+             }
+
+             if (hasFaceNormal) {
+                 normalIndex = faces[offset++] * 3
+
+                 faceA.normal.set(
+                     normals[normalIndex++],
+                     normals[normalIndex++],
+                     normals[normalIndex]
+                 )
+                 faceB.normal.copy(faceA.normal)
+             }
+
+             if (hasFaceVertexNormal) {
+                 for (i = 0; i < 4; i++) {
+                     normalIndex = faces[offset++] * 3
+                     normal = new Vector3(
+                         normals[normalIndex++],
+                         normals[normalIndex++],
+                         normals[normalIndex]
+                     )
+
+                     if (i !== 2) faceA.vertexNormals.push(normal)
+                     if (i !== 0) faceB.vertexNormals.push(normal)
+                 }
+             }
+
+             if (hasFaceColor) {
+                 colorIndex = faces[offset++]
+                 hex = colors[colorIndex]
+                 faceA.color.setHex(hex)
+                 faceB.color.setHex(hex)
+             }
+
+             if (hasFaceVertexColor) {
+                 for (i = 0; i < 4; i++) {
+                     colorIndex = faces[offset++]
+                     hex = colors[colorIndex]
+
+                     if (i !== 2) faceA.vertexColors.push(new Color(hex))
+                     if (i !== 0) faceB.vertexColors.push(new Color(hex))
+                 }
+             }
+
+             geometry.faces.push(faceA)
+             geometry.faces.push(faceB)
+         } else {
+             face = new Face3()
+             face.a = faces[offset++]
+             face.b = faces[offset++]
+             face.c = faces[offset++]
+
+             if (hasMaterial) {
+                 materialIndex = faces[offset++]
+                 face.materialIndex = materialIndex
+             }
+
+             fi = geometry.faces.length
+
+             if (hasFaceVertexUv) {
+                 for (i = 0; i < nUvLayers; i++) {
+                     uvLayer = json.uvs[i]
+                     geometry.faceVertexUvs[i][fi] = []
+
+                     for (j = 0; j < 3; j++) {
+                         uvIndex = faces[offset++]
+                         u = uvLayer[uvIndex * 2]
+                         v = uvLayer[uvIndex * 2 + 1]
+                         uv = new Vector2(u, v)
```

```
+                        geometry.faceVertexUvs[i][fi].push(uv)
+                    }
+                }
+            }
+
+            if (hasFaceNormal) {
+                normalIndex = faces[offset++] * 3
+                face.normal.set(
+                    normals[normalIndex++],
+                    normals[normalIndex++],
+                    normals[normalIndex]
+                )
+            }
+
+            if (hasFaceVertexNormal) {
+                for (i = 0; i < 3; i++) {
+                    normalIndex = faces[offset++] * 3
+                    normal = new Vector3(
+                        normals[normalIndex++],
+                        normals[normalIndex++],
+                        normals[normalIndex]
+                    )
+                    face.vertexNormals.push(normal)
+                }
+            }
+
+            if (hasFaceColor) {
+                colorIndex = faces[offset++]
+                face.color.setHex(colors[colorIndex])
+            }
+
+            if (hasFaceVertexColor) {
+                for (i = 0; i < 3; i++) {
+                    colorIndex = faces[offset++]
+                    face.vertexColors.push(new Color(colors[colorIndex]))
+                }
+            }
+
+            geometry.faces.push(face)
+        }
+    }
+ }
+
+ function parseSkin(json, geometry) {
+     const influencesPerVertex = json.influencesPerVertex !== undefined ? json.influencesPerVertex : 2
+
+     if (json.skinWeights) {
+         for (let i = 0, l = json.skinWeights.length; i < l; i += influencesPerVertex) {
+             const x = json.skinWeights[i]
+             const y = influencesPerVertex > 1 ? json.skinWeights[i + 1] : 0
+             const z = influencesPerVertex > 2 ? json.skinWeights[i + 2] : 0
+             const w = influencesPerVertex > 3 ? json.skinWeights[i + 3] : 0
+             geometry.skinWeights.push(new Vector4(x, y, z, w))
+         }
+     }
+
+     if (json.skinIndices) {
+         for (let i = 0, l = json.skinIndices.length; i < l; i += influencesPerVertex) {
+             const a = json.skinIndices[i]
+             const b = influencesPerVertex > 1 ? json.skinIndices[i + 1] : 0
+             const c = influencesPerVertex > 2 ? json.skinIndices[i + 2] : 0
+             const d = influencesPerVertex > 3 ? json.skinIndices[i + 3] : 0
+             geometry.skinIndices.push(new Vector4(a, b, c, d))
+         }
+     }
+
+     geometry.bones = json.bones
+ }
+
```

```
+ function parseMorphing(json, geometry) {
+     const scale = json.scale || 1
+
+     if (json.morphTargets !== undefined) {
+         for (let i = 0, l = json.morphTargets.length; i < l; i++) {
+             geometry.morphTargets[i] = {}
+             geometry.morphTargets[i].name = json.morphTargets[i].name
+             geometry.morphTargets[i].vertices = []
+
+             const dstVertices = geometry.morphTargets[i].vertices
+             const srcVertices = json.morphTargets[i].vertices
+
+             for (let v = 0, vl = srcVertices.length; v < vl; v += 3) {
+                 const vertex = new Vector3()
+                 vertex.x = srcVertices[v] * scale
+                 vertex.y = srcVertices[v + 1] * scale
+                 vertex.z = srcVertices[v + 2] * scale
+                 dstVertices.push(vertex)
+             }
+         }
+     }
+
+     if (json.morphColors !== undefined && json.morphColors.length > 0) {
+         const faces = geometry.faces
+         const morphColors = json.morphColors[0].colors
+
+         for (let i = 0, l = faces.length; i < l; i++) {
+             faces[i].color.fromArray(morphColors, i * 3)
+         }
+     }
+ }
+
+ function toAbsolutePath(path, texturePath) {
+     if (!path) return path
+     if (/^(https?:)?\/\//i.test(path)) return path
+     if (path.startsWith('data:')) return path
+     if (!texturePath) return path
+     return `${texturePath}${path}`.replace(/\\/g, '/')
+ }
+
+ function applyLegacyMaterialProps(material, jsonMaterial) {
+     if (!jsonMaterial) return material
+
+     if (jsonMaterial.DbgName) material.name = jsonMaterial.DbgName
+     if (jsonMaterial.name) material.name = jsonMaterial.name
+
+     if (Array.isArray(jsonMaterial.colorDiffuse)) material.color.fromArray(jsonMaterial.colorDiffuse)
+     if (Array.isArray(jsonMaterial.colorSpecular)) material.specular.fromArray(jsonMaterial.colorSpecular)
+     if (Array.isArray(jsonMaterial.colorEmissive)) material.emissive.fromArray(jsonMaterial.colorEmissive)
+
+     if (jsonMaterial.specularCoef !== undefined) material.shininess = Number(jsonMaterial.specularCoef) * 100
+     if (jsonMaterial.opacity !== undefined) material.opacity = Number(jsonMaterial.opacity)
+     if (jsonMaterial.transparent !== undefined) material.transparent = Boolean(jsonMaterial.transparent)
+     if (jsonMaterial.wireframe !== undefined) material.wireframe = Boolean(jsonMaterial.wireframe)
+
+     if (material.opacity < 1) material.transparent = true
+
+     return material
+ }
+
+ function loadOptionalTexture(textureLoader, texturePath, textureFile) {
+     if (!textureFile) return null
+     const resolved = toAbsolutePath(textureFile, texturePath)
+     return textureLoader.load(resolved)
+ }
+
+ function buildMaterials(jsonMaterials, texturePath) {
+     if (!Array.isArray(jsonMaterials) || jsonMaterials.length === 0) {
+         return []
```

```
+     }
+
+     const textureLoader = new TextureLoader()
+
+     return jsonMaterials.map((jsonMaterial, i) => {
+         const material = applyLegacyMaterialProps(new MeshPhongMaterial(), jsonMaterial)
+         if (!material.name) material.name = `material_${i}`
+
+         const map = loadOptionalTexture(textureLoader, texturePath, jsonMaterial.mapDiffuse)
+         if (map) material.map = map
+
+         const alphaMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial.mapAlpha)
+         if (alphaMap) {
+             material.alphaMap = alphaMap
+             material.transparent = true
+         }
+
+         const bumpMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial.mapBump)
+         if (bumpMap) material.bumpMap = bumpMap
+
+         const normalMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial.mapNormal)
+         if (normalMap) material.normalMap = normalMap
+
+         const specularMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial.mapSpecular)
+         if (specularMap) material.specularMap = specularMap
+
+         material.skinning = true
+         material.morphTargets = true
+         return material
+     })
+ }
+
+ function ensureMaterialArray(materials) {
+     const materialArray = Array.isArray(materials) ? materials : [materials]
+     materialArray.materials = materialArray
+     return materialArray
+ }
+
+ function defineLegacyAlias(bufferGeometry, legacyGeometry, key) {
+     Object.defineProperty(bufferGeometry, key, {
+         configurable: true,
+         enumerable: false,
+         get() {
+             return legacyGeometry[key]
+         },
+         set(value) {
+             legacyGeometry[key] = value
+         }
+     })
+ }
+
+ export function extractUrlBase(url) {
+     return LoaderUtils.extractUrlBase(url)
+ }
+
+ export function createLegacyMaterial(jsonMaterial, texturePath = '') {
+     const material = applyLegacyMaterialProps(new MeshPhongMaterial(), jsonMaterial || {})
+     const textureLoader = new TextureLoader()
+
+     const map = loadOptionalTexture(textureLoader, texturePath, jsonMaterial && jsonMaterial.mapDiffuse)
+     if (map) material.map = map
+
+     const alphaMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial && jsonMaterial.mapAlpha)
+     if (alphaMap) {
+         material.alphaMap = alphaMap
+         material.transparent = true
+     }
+
+     const bumpMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial && jsonMaterial.mapBump)
```

```
+     if (bumpMap) material.bumpMap = bumpMap
+
+     const normalMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial && jsonMaterial.mapNormal)
+     if (normalMap) material.normalMap = normalMap
+
+     const specularMap = loadOptionalTexture(textureLoader, texturePath, jsonMaterial && jsonMaterial.mapSpecular)
+     if (specularMap) material.specularMap = specularMap
+
+     material.skinning = true
+     material.morphTargets = true
+     return material
+ }
+
+ export function parseLegacyModel(json, texturePath = '') {
+     if (json && json.data !== undefined) {
+         json = json.data
+     }
+
+     if (!json) {
+         return { legacyGeometry: new LegacyGeometry(), materials: [] }
+     }
+
+     json.scale = json.scale !== undefined ? 1.0 / json.scale : 1.0
+
+     const legacyGeometry = new LegacyGeometry()
+     parseModel(json, legacyGeometry)
+     parseSkin(json, legacyGeometry)
+     parseMorphing(json, legacyGeometry)
+
+     legacyGeometry.computeFaceNormals()
+     legacyGeometry.computeBoundingSphere()
+
+     const materials = buildMaterials(json.materials, texturePath)
+     return { legacyGeometry, materials }
+ }
+
+ export function attachLegacyGeometry(bufferGeometry, legacyGeometry) {
+     bufferGeometry._legacyGeometry = legacyGeometry
+     bufferGeometry._bufferGeometry = bufferGeometry
+
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'vertices')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'faces')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'faceVertexUvs')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'morphTargets')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'skinWeights')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'skinIndices')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'lineDistances')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'elementsNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'verticesNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'uvsNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'normalsNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'colorsNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'lineDistancesNeedUpdate')
+     defineLegacyAlias(bufferGeometry, legacyGeometry, 'groupsNeedUpdate')
+
+     return bufferGeometry
+ }
+
+ function ensurePositionAttribute(bufferGeometry, vertexCount) {
+     let position = bufferGeometry.getAttribute('position')
+     if (!position || position.count !== vertexCount) {
+         position = new Float32BufferAttribute(vertexCount * 3, 3)
+         bufferGeometry.setAttribute('position', position)
+     }
+     return position
+ }
+
+ function ensureUvAttribute(bufferGeometry, uvCount) {
+     let uv = bufferGeometry.getAttribute('uv')
```

```
+     if (!uv || uv.count !== uvCount) {
+         uv = new Float32BufferAttribute(uvCount * 2, 2)
+         bufferGeometry.setAttribute('uv', uv)
+     }
+     return uv
+ }
+
+ function ensureIndexAttribute(bufferGeometry, vertexCount, indexCount) {
+     const use32Bit = vertexCount > 65535
+     const indexCtor = use32Bit ? Uint32Array : Uint16Array
+     let index = bufferGeometry.getIndex()
+
+     if (!index || index.count !== indexCount || !(index.array instanceof indexCtor)) {
+         index = new BufferAttribute(new indexCtor(indexCount), 1)
+         bufferGeometry.setIndex(index)
+     }
+
+     return index
+ }
+
+ function syncSkinData(bufferGeometry, legacyGeometry) {
+     if (!legacyGeometry.skinWeights || legacyGeometry.skinWeights.length === 0) return
+
+     const skinWeights = new Float32Array(legacyGeometry.skinWeights.length * 4)
+     const skinIndices = new Float32Array(legacyGeometry.skinIndices.length * 4)
+
+     for (let i = 0; i < legacyGeometry.skinWeights.length; i++) {
+         const w = legacyGeometry.skinWeights[i]
+         const idx = legacyGeometry.skinIndices[i]
+
+         skinWeights[i * 4] = w.x
+         skinWeights[i * 4 + 1] = w.y
+         skinWeights[i * 4 + 2] = w.z
+         skinWeights[i * 4 + 3] = w.w
+
+         skinIndices[i * 4] = idx.x
+         skinIndices[i * 4 + 1] = idx.y
+         skinIndices[i * 4 + 2] = idx.z
+         skinIndices[i * 4 + 3] = idx.w
+     }
+
+     bufferGeometry.setAttribute('skinWeight', new Float32BufferAttribute(skinWeights, 4))
+     bufferGeometry.setAttribute('skinIndex', new Float32BufferAttribute(skinIndices, 4))
+ }
+
+ export function syncLegacyGeometry(bufferGeometry) {
+     const legacyGeometry = bufferGeometry && bufferGeometry._legacyGeometry
+     if (!legacyGeometry) return bufferGeometry
+
+     const vertices = legacyGeometry.vertices || []
+     const faces = legacyGeometry.faces || []
+     const uvs = (legacyGeometry.faceVertexUvs && legacyGeometry.faceVertexUvs[0]) || []
+
+     const position = ensurePositionAttribute(bufferGeometry, vertices.length)
+     for (let i = 0; i < vertices.length; i++) {
+         const v = vertices[i]
+         position.setXYZ(i, v.x, v.y, v.z)
+     }
+     position.needsUpdate = true
+
+     if (faces.length > 0) {
+         const index = ensureIndexAttribute(bufferGeometry, vertices.length, faces.length * 3)
+
+         for (let i = 0; i < faces.length; i++) {
+             const face = faces[i]
+             index.setX(i * 3, face.a)
+             index.setX(i * 3 + 1, face.b)
+             index.setX(i * 3 + 2, face.c)
+         }
```

```
+         index.needsUpdate = true
+
+         bufferGeometry.clearGroups()
+
+         let groupStart = 0
+         let currentMaterial = faces[0].materialIndex || 0
+         for (let i = 1; i < faces.length; i++) {
+             const materialIndex = faces[i].materialIndex || 0
+             if (materialIndex !== currentMaterial) {
+                 bufferGeometry.addGroup(groupStart * 3, (i - groupStart) * 3, currentMaterial)
+                 groupStart = i
+                 currentMaterial = materialIndex
+             }
+         }
+         bufferGeometry.addGroup(groupStart * 3, (faces.length - groupStart) * 3, currentMaterial)
+     }
+
+     if (uvs.length === faces.length && faces.length > 0) {
+         const uv = ensureUvAttribute(bufferGeometry, faces.length * 3)
+         for (let i = 0; i < faces.length; i++) {
+             const triUvs = uvs[i]
+             if (!triUvs || triUvs.length < 3) continue
+             uv.setXY(i * 3, triUvs[0].x, triUvs[0].y)
+             uv.setXY(i * 3 + 1, triUvs[1].x, triUvs[1].y)
+             uv.setXY(i * 3 + 2, triUvs[2].x, triUvs[2].y)
+         }
+         uv.needsUpdate = true
+     }
+
+     syncSkinData(bufferGeometry, legacyGeometry)
+
+     bufferGeometry.computeVertexNormals()
+     bufferGeometry.computeBoundingBox()
+     bufferGeometry.computeBoundingSphere()
+     legacyGeometry.boundingBox = bufferGeometry.boundingBox ? bufferGeometry.boundingBox.clone() : null
+     legacyGeometry.boundingSphere = bufferGeometry.boundingSphere ? bufferGeometry.boundingSphere.clone() : null
+     return bufferGeometry
+ }
+
+ export function legacyToBufferGeometry(legacyGeometry) {
+     const bufferGeometry = new BufferGeometry()
+     attachLegacyGeometry(bufferGeometry, legacyGeometry)
+     syncLegacyGeometry(bufferGeometry)
+     return bufferGeometry
+ }
+
+ export function createMaterialArray(materials) {
+     return ensureMaterialArray(materials)
+ }
+
+ export function loadTextFile(url, manager) {
+     const loader = new FileLoader(manager)
+     return new Promise((resolve, reject) => {
+         loader.load(url, resolve, undefined, reject)
+     })
+ }
```

FILE: src/js/human/human.js

Status: MODIFIED | +139 / -35

```
    /**
     * @name            MakeHuman
     * @copyright       MakeHuman Team 2001-2016
     * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
     * @author          wassname
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     * @description
```

```
 * Describes human class which holds the 3d mesh, modifiers, human factors,
 * and morphTargets.
 */

import _ from 'lodash'
import * as THREE from 'three'
import TWEEN from 'tween'

import * as qs from 'qs'
import Targets from './targets'
import Modifiers from './human-modifier'
- import poses from 'makehuman-data/src/json/poses/poses.json'
import { Proxies } from './proxy'
import { EthnicSkinBlender } from './ethnic-skin-blender'
import Factors from './factors'
import { deepParseFloat, remapKeyValuesDeep, deepRoundValues } from '../helpers/helpers'
+ import {
+     createLegacyMaterial,
+     createMaterialArray,
+     extractUrlBase,
+     legacyToBufferGeometry,
+     loadTextFile,
+     parseLegacyModel,
+     syncLegacyGeometry
+ } from '../helpers/legacy-three-compat'
import '../helpers/OBJExporter'
// import SubdivisionModifier from
"imports?THREE=three!exports?THREE.SubdivisionModifier!three/examples/js/modifiers/SubdivisionModifier";
// import BufferSubdivisionModifier from
"imports?THREE=three!exports?THREE.SubdivisionModifier!three/examples/js/modifiers/BufferSubdivisionModifier";
import OBJExporter from '../helpers/OBJExporter'

+ let posesPromise = null
+ function loadPoses() {
+     if (!posesPromise) {
+         posesPromise = import(/* webpackChunkName: "poses-data" */ 'makehuman-data/src/json/poses/poses.json')
+             .then(module => module.default || module)
+             .catch((error) => {
+                 console.warn('Failed to lazy-load poses.json, continuing with empty poses set.', error)
+                 return {}
+             })
+     }
+     return posesPromise
+ }

+ function buildSkeletonFromLegacyBones(mesh, legacyGeometry) {
+     const boneData = legacyGeometry && legacyGeometry.bones
+     if (!Array.isArray(boneData) || boneData.length === 0) {
+         return null
+     }

+     const bones = boneData.map((data, index) => {
+         const bone = new THREE.Bone()
+         bone.name = data && data.name ? data.name : `bone_${index}`
+
+         if (data && Array.isArray(data.pos)) {
+             bone.position.fromArray(data.pos)
+         }
+         if (data && Array.isArray(data.rotq)) {
+             bone.quaternion.fromArray(data.rotq)
+         }
+         if (data && Array.isArray(data.scl)) {
+             bone.scale.fromArray(data.scl)
+         }
+
+         return bone
+     })
+
+     for (let i = 0; i < boneData.length; i++) {
+         const parentIndex = boneData[i] && Number.isInteger(boneData[i].parent) ? boneData[i].parent : -1
```

```
+          if (parentIndex >= 0 && bones[parentIndex]) {
+              bones[parentIndex].add(bones[i])
+          } else {
+              mesh.add(bones[i])
+          }
+      }
+
+      const skeleton = new THREE.Skeleton(bones)
+      mesh.bind(skeleton)
+      if (typeof mesh.normalizeSkinWeights === 'function') {
+          mesh.normalizeSkinWeights()
+      }
+      return skeleton
+  }
+
+
+
    export class HumanIO {
        constructor(human) {
            this.human = human

            this.rounding = 3

            const modifierFullNames = Object.keys(this.human.modifiers.children).sort()
            const modifierFullNameMapping = _.fromPairs(_.map(modifierFullNames, (k, v) => [k, v]))
            this.shortenMapping = { poseName: 'pn', skin: 's', modifiers: 'm', proxies: 'p', ...modifierFullNameMapping }
        }

        toConfig() {
            return this.human.exportConfig()
        }

        fromConfig(config) {
            this.human.importConfig(config)
        }

        toShortConfig(config = this.human.exportConfig()) {
            config = remapKeyValuesDeep(config, this.shortenMapping, {})
            config = deepRoundValues(config, v => _.round(v, this.rounding))
            return config
        }

        fromShortConfig(sortConfig) {
            return remapKeyValuesDeep(sortConfig, _.invert(this.shortenMapping), {})
        }

        fromUrlQuery(queryConfig) {
            const config = qs.parse(queryConfig)
            return deepParseFloat(remapKeyValuesDeep(config, _.invert(this.shortenMapping), {}))
        }

        /* transforms the config to a smaller url query **/
        toUrlQuery(config = this.human.exportConfig(), encode = true) {
            config = remapKeyValuesDeep(config, this.shortenMapping, {})
            config = deepRoundValues(config, v => _.round(v, this.rounding))
            const queryConfig = qs.stringify(config, { encode })
            if (queryConfig.length < 2048) throw new Error('url config should be shorter than 2048 chars')
            return queryConfig
        }

        toUrl(config = this.human.exportConfig(), encode = true) {
            return `${window.location.origin}?${this.toUrlQuery(config, encode)}`
        }

        fromUrl(url = window.location.toString()) {
            const parser = document.createElement('a')
            parser.href = url
            const config = this.fromUrlQuery(parser.search.slice(1))
            this.human.importConfig(config)
```

```
            return config
        }

        /**
         * Export the human mesh with morphs but not pose, skin, or accessories.
         * @param {bool} helpers - if true it strips the helper meshes like hair-helper.
         * @return {string} Wavefront obj file compatible with blender
         */
        toObj(helpers=false) {
            // const self = this
            const objExporter = new OBJExporter()

            const mesh = this.human.mesh.clone()
-           mesh.geometry = mesh.geometry.clone()
+           if (this.human.geometry && this.human.geometry.clone) {
+               mesh.geometry = this.human.geometry.clone()
+           } else {
+               mesh.geometry = mesh.geometry.clone()
+           }
+           const sourceMaterials = this.human.mesh.material.materials || this.human.mesh.material || []
+           mesh.material = createMaterialArray(sourceMaterials.map(m => (m && m.clone ? m.clone() : m)))

            mesh.name = `makehuman_1.1-${new Date().toJSON()}`

            // unmask vertices under clothes
            const nullMaterial = mesh.material.materials.findIndex(m => m.name == "maskedFaces")
            mesh.geometry.faces.forEach((f, i) => {
                if (f.materialIndex === nullMaterial) {
                    f.materialIndex = this.human.mesh.geometry.faces[i].oldMaterialIndex
                }
            })

            if (!helpers) {
                /* Makehuman has a human body mesh, then it has some invisible meshes attached to it.
                 * These are hair-helper, dress-helpers etc which invisible extensions to the human body
                 * used to attach clothes or hair to. When the human is morphed so are the helpers,
                 * ensuring that the clothes fit the morphed human.
                 * If the helper option is not selected, lets remove the helper vertices.
                 */
                const geom = mesh.geometry

                // delete unused, uvs, faces, and vertices
                geom.faceVertexUvs = geom.faceVertexUvs.filter((uv, i) => geom.faces[i].materialIndex === 0)
                geom.faces = geom.faces.filter(f => f.materialIndex === 0)

                // TODO remove unused vertices without breaking the obj
                const verticesToKeep = _.sortBy(_.uniq(_.concat(
                    ...geom.faces.filter(f => f.materialIndex === 0)
                    .map(f => [f.a, f.b, f.c])
                )))
                geom.vertices = geom.vertices.filter((v, i) => verticesToKeep.includes(i))
                geom.faces.forEach((f) => {
                    f.a = verticesToKeep.indexOf(f.a)
                    f.b = verticesToKeep.indexOf(f.b)
                    f.c = verticesToKeep.indexOf(f.c)
                })
            }

            let obj = objExporter.parse(mesh)
            // don't export vertex normals
            obj = obj.split('\n').filter(line => !line.startsWith('vn ')).join('\n')

            // header data
            const jsonMetadata = JSON.stringify(this.human.metadata, null, 4).replace(/\n/g, '\n#')
            const header = `# Exported from makehuman js on ${new Date().toJSON()}\n#Source
metadata:\n#${jsonMetadata}\n`

            return header + obj
        }
    }
```

```
/**
 * Basic human with method to load base mash, skins, and config
 * @type {Class}
 */
export class BaseHuman extends THREE.Object3D {
    /**
      * @param  {Object[]} config [description]
      * @param  {String} config[].x - X position
      * @param  {String} config[].y - Y position
      * @param  {String} config[].y - Z position
      * @param  {String} config[].skins - Url to skins
      * @param  {String} config[].character - Url to body json suitable for json
      *                                       loader
      * @param  {String} config[].baseUrl - Baseurl
     */
    constructor(config) {
        super()

        this.config = config = _.defaults(config, {
            skins: [],
            proxies: [],
            poses: [],
            targets: 'targets.bin',
            model: 'base.json',
            baseUrl: "data/",
            x: 0,
            y: 0,
            z: 0,
            s: 1
        })

        this.mesh = null
        this.skins = []

-         this.poses = poses
+         this.poses = {}
+         this._posesLoaded = false
+         this._posesReady = loadPoses()
+             .then((loadedPoses) => {
+                 this.poses = loadedPoses || {}
+                 this._posesLoaded = true
+                 return this.poses
+             })
+             .catch(() => {
+                 this.poses = {}
+                 this._posesLoaded = true
+                 return this.poses
+             })

        // TODO put in config
        this.minUpdateInterval = 1000 // minimum interval to recalc vertices in ms

        // TODO undo this hardcoding
        // this.skeleton_metadata = skeleton_metadata

        this._poseTweens = []
        this._skinCache = {}

        // this.mixer = new THREE.AnimationMixer(this);

        this.manager = new THREE.LoadingManager()
        this.manager.onLoad = this.onLoadComplete.bind(this)
        this.materialLoader = new THREE.MaterialLoader(this.manager)

        this.onBeforeRender = BaseHuman.prototype.onBeforeRender
        this.onAfterRender = BaseHuman.prototype.onAfterRender
    }
```

```
        /**
         * Loads body from config
         * @return {Promise}                - promise of loaded human
         */
        loadModel() {
            const self = this
            const config = this.config

            // HUMAN
            // Load the geometry data from a url
-           this.loader = new THREE.XHRLoader(this.manager)
+           this.loader = new THREE.FileLoader(this.manager)
            const modelUrl = `${config.baseUrl}models/${config.model}`
            return new Promise((resolve, reject) => {
                try {
                    self.loader.load(modelUrl,
                            resolve,
                            undefined,
                            reject
                        )
                } catch (e) {
                    reject(e)
                }
            })
                .catch((err) => {
                    console.error('Failed to load model data', modelUrl, err)
                    throw err
                })
                .then(text =>
                    JSON.parse(text)
                )
                .then((json) => {
                    self.metadata = json.metadata
-                   const texturePath = self.texturePath && (typeof self.texturePath === "string") ? self.texturePath : THREE.Loader.prototype.extractUrlBase(modelUrl)
-
-                   return new THREE.JSONLoader().parse(json, texturePath)
+                   const texturePath = self.texturePath && (typeof self.texturePath === "string") ? self.texturePath : extractUrlBase(modelUrl)
+                   return parseLegacyModel(json, texturePath)
                })
                // use unpacking here to turn one args into two, as promises only return one
                .then(({
-                   geometry,
+                   legacyGeometry,
                    materials
                }) => {
-                   self.geometry = geometry
+                   self.geometry = legacyGeometry

-                   geometry.computeBoundingBox()
+                   legacyGeometry.computeBoundingBox()
                    // geometry.computeVertexNormals()
-                   geometry.name = config.character
+                   legacyGeometry.name = config.character

                    materials.map(m => (m.morphTargets = true))
                    materials.map(m => (m.skinning = true))

                    // add a null material, and backup face materialIndexes
-                   geometry.faces.map(face => (face.oldMaterialIndex = face.materialIndex))
+                   legacyGeometry.faces.map(face => (face.oldMaterialIndex = face.materialIndex))
                    materials.push(new THREE.MeshBasicMaterial({ visible: false, name: 'maskedFaces' }))

                    // load multiple materials, to group helper faces using materials http://stackoverflow.com/questions/11025307/can-i-hide-faces-of-a-mesh-in-three-js
-                   self.mesh = new THREE.SkinnedMesh(geometry, new THREE.MultiMaterial(materials))
+                   const materialArray = createMaterialArray(materials)
+                   self.mesh = new THREE.SkinnedMesh(legacyToBufferGeometry(legacyGeometry), materialArray)
                    self.mesh.name = config.character
```

```
                self.add(self.mesh)

-               this.skeleton = this.mesh.skeleton
+               this.skeleton = buildSkeletonFromLegacyBones(self.mesh, legacyGeometry) || self.mesh.skeleton

                self.scale.set(config.s, config.s, config.s)

                self.mesh.geometry.computeBoundingBox()
-               const halfHeight = self.mesh.geometry.boundingBox.getSize().y / 2
+               const halfHeight = self.mesh.geometry.boundingBox.getSize(new THREE.Vector3()).y / 2
                self.position.set(config.x, config.y + halfHeight, config.z)

                self.mesh.castShadow = true
                self.mesh.receiveShadow = true

+               syncLegacyGeometry(self.mesh.geometry)

-               self.mesh.geometry.computeVertexNormals()
-
                self.updateJointPositions()

                // hide the helper parts
                self.bodyPartOpacity(0)
                return self.setSkin(config.defaultSkin)
                    .then(() => self)
            })
    }

    /**
     * Load targets from .target urls
     * @param  {String}      dataUrl    Url of the targt binary
     * @return {Promise}                Promise of an array of targets
     */
    loadTargets(dataUrl=this.config.baseUrl+'targets/'+this.config.targets) {
        return this.targets.load(dataUrl)
            .then(targets => targets)
    }

    onLoadComplete() {}

    /**
     * This sets the opacity of parts of the body.
     * With name given arguments it sets all helpers and joints
     * If no arguments are given it lists helper names.
     * @param  {Number} opacity - Set opacity of the helper/s to this
     * @param  {String} name    - Optional helper, otherwise all helpers are set
     * @return {Number}         - Amount of helper opacities set
     */
    bodyPartOpacity(opacity, name) {
        let parts

        // return lists of parts
        if (opacity === undefined) {
            return this.mesh.material.materials.map(m => m.name)
        }

        const helpersAndJoints = this.mesh.material.materials.filter(m => typeof m.name === 'string' &&
                (m.name.startsWith('joint') || m.name.startsWith('helper')))

        if (name) {
            parts = this.mesh.material.materials.filter(m => m.name === name)
        } else {
            parts = helpersAndJoints
        }
        for (let i = 0; i < parts.length; i++) {
            // no point in rendering it at 0 opacity
            if (opacity === 0) {
                parts[i].visible = false
            } else { parts[i].visible = true }
```

```
            parts[i].opacity = opacity
            parts[i].transparent = opacity < 1
        }
        return parts.length
    }

    /** Set this bodies texture map from a loaded skin material **/
    setSkin(url = this.config.defaultSkin) {
        const base = `${this.config.baseUrl}skins/`
        if (!url.startsWith(base)) {
            url = `${this.config.baseUrl}skins/${url}`
        }
        return Promise.resolve().then(() => {
            if (this._skinCache[url]) {
                return this._skinCache[url]
            } else {
-                return new Promise((resolve, reject) => {
-                    this.loader.load(url, resolve, undefined, reject)
-                })
+                return loadTextFile(url, this.manager)
                 .then((text) => {
                     const json = JSON.parse(text)
-                    const texturePath = self.texturePath &&
-                        (typeof self.texturePath === "string") ?
-                            self.texturePath :
-                            THREE.Loader.prototype.extractUrlBase(url)
-                    return new THREE.Loader().createMaterial(json, texturePath)
+                    const texturePath = this.texturePath &&
+                        (typeof this.texturePath === "string") ?
+                            this.texturePath :
+                            extractUrlBase(url)
+                    return createLegacyMaterial(json, texturePath)
                 })
                 .then((material) => {
                     this._skinCache[url] = material
                     material.name = url.split('/').slice(-1)[0].split('.')[0]
                     material.skinning = true
                     return material
                 })
            }
        })
        .then((material) => {
            if (this.mesh && this.mesh.material.materials) {
-                return this.mesh.material.materials[0] = material
+                this.mesh.material.materials[0] = material
+                this.mesh.material[0] = material
+                return true
            } else {
                return false
            }
        })
    }

    /**
     *  Calculate the position of specified named joint from the current
     *  state of the human mesh. If this skeleton contains no vertex mapping
     *  for that joint name, it falls back to looking for a vertex group in the
     *  human basemesh with that joint name.
     * @type {String} - bone name e.g. head
     */
    getJointPosition(boneName, head = false) {
        // TODO if inRest then get geom, else buffer geom
        if (boneName && boneName.indexOf('____head') === -1 && boneName.indexOf('____tail') === -1) { boneName += head ? '____head' : '____tail' }
        // let bone_id = _.findIndex(this.skeleton.bones, bone=>bone.name==boneName)
        const vertices = this.metadata.joint_pos_idxs[boneName]
        if (vertices) {
            const positions = vertices
                .map(vId => this.mesh.geometry.vertices[vId].clone())
```

```
                return new THREE.Vector3(
                    _.mean(positions.map(v => v.x)),
                    _.mean(positions.map(v => v.y)),
                    _.mean(positions.map(v => v.z))
                )
            } else {
                return null
            }
        }

        /**
         * We move bones towards the vertices they are weighted to, adjusting for
         * change in mesh size
         * See makehumans shared/skeleton.py:Skeleton:updateJointPositions
         */
        updateJointPositions() {
            const skeleton = this.skeleton
+           if (!skeleton || !Array.isArray(skeleton.bones) || skeleton.bones.length === 0) {
+               return false
+           }
            const self = this
            const identity = new THREE.Matrix4().identity()

            // undo pose, while we do this
            const poseName = self._poseName
            self.setPose()

            // get positions of reference vertices
            // first get positions of bones, as we don't want changes to propogate to the mesh then to the skeleton
            const positions = skeleton.bones.map((bone) => {
                const vTail = this.getJointPosition(bone.name, false, true)
                const vHead = this.getJointPosition(bone.parent.name, false, true)
                if (vTail && vHead) {
                    return vTail.clone().sub(vHead)
                }
                if (vTail) {
                    return vTail
                } else {
                    console.warn(`couldn't update ${bone.name} because no weights or group for ${vTail ? bone.name :
    bone.parent.name}`)
                    return null
                }
            })

            // now update referenceMatrix
            for (let i = 0; i < skeleton.bones.length; i++) {
                const boneInverse = new THREE.Matrix4()
                const bone = skeleton.bones[i]
                const parentIndex = _.findIndex(skeleton.bones, b => b.name === bone.parent.name)
                const parent = skeleton.bones[parentIndex]
                const position = positions[i]

                if (position) {
                    bone.position.set(position.x, position.y, position.z)
                    bone.updateMatrixWorld()

                    if (i > 0) {
-                       boneInverse.getInverse(parent.matrixWorld)
+                       boneInverse.copy(parent.matrixWorld).invert()
                            .multiply(bone.matrixWorld)
                        // subtract parents
                        for (let j = 0; j < boneInverse.elements.length; j++) {
                            boneInverse.elements[j] = skeleton.boneInverses[parentIndex].elements[j] -
    boneInverse.elements[j] + identity.elements[j]
                        }
                    } else {
                        boneInverse
-                           .getInverse(bone.matrixWorld).multiply(self.matrixWorld)
+                           .copy(bone.matrixWorld).invert().multiply(self.matrixWorld)
                    }
```

```
                    // TODO make into test, it shouldn't change boneInverses on initial load with no pose
                    // let diffs =_.zipWith(
                    //     boneInverse.elements,
                    //     self.mesh.skeleton.boneInverses[i].elements,
                    //     (a,b)=>a-b
                    // )
                    // console.assert(
                    //     diffs.filter(d=>d>0.001) .length==0,'in pose position these should be equal '+bone.name+'
'+diffs
                    // )
                    skeleton.boneInverses[i] = boneInverse
                }
            }
            // FIXME there should be a way of doing this without actually changing bone positions, the rePosing but it
works for now
            self.setPose(poseName)
        }


        setPose(poseName, interval = 0) {
+           if (poseName && !this._posesLoaded && this._posesReady) {
+               return this._posesReady.then(() => this.setPose(poseName, interval))
+           }
+
+           if (!this.mesh || !this.mesh.skeleton || !Array.isArray(this.mesh.skeleton.bones)) {
+               this._poseName = null
+               return false
+           }
+
            // TODO, load each from json like a proxy
            const pose = this.poses[poseName]
            const self = this
            if (pose) {
                this._poseName = poseName
                self._poseTweens.map((tween) => { return tween ? tween.stop() : '' })
                self._poseTweens = Object.keys(pose).map((boneName) => {
                    const bone = this.mesh.skeleton.bones.find(b => b.name === boneName)
                    if (!bone) {
                        console.error('couldnt find bone', boneName)
                        return null
                    }
                    // Tween the pose
                    const data = pose[bone.name]
                    if (!interval) {
                        // skip the tween at interval zero
                        // we apply the rotation to the one above it (the head), since it modifies the one after
                        bone.parent.quaternion.set(...data)
                        return null
                    } else {
                        const qBefore = bone.parent.quaternion.clone()
                        const qAfter = new THREE.Quaternion().set(...data)
                        const t = 0
                        return new TWEEN.Tween(t)
                        .to(1, interval)
                        .onUpdate((ti) => {
-                           THREE.Quaternion.slerp(qBefore, qAfter, bone.parent.quaternion, ti)
+                           bone.parent.quaternion.copy(qBefore).slerp(qAfter, ti)
                        })
                        .start()
                    }
                })
            } else {
-               this.mesh.pose()
+               if (typeof this.mesh.pose === 'function') {
+                   this.mesh.pose()
+               }
                this._poseName = null
            }
            // this.updateHeight()
```

```
    }

    onElementsNeedUpdate() {}

    onBeforeRender() {
    }

    onAfterRender() {
    }

    exportConfig() {
        // consider doing bodyPartOpacity, consider using this.toJSON
        // also consifer exporting config althoug we might need to reload human then
        const json = {
            skin: this.mesh.material.materials[0].name,
            poseName: this._poseName
        }
        return json
    }

    importConfig(json) {
        if (json.skin) {
            this.setSkin(json.skin)
        }
        if (json.poseName) {
            this.setPose(json.poseName)
        }
    }
}

/**
 * Extends BaseHuman to have targets and modifiers to manage them
 */
export class Human extends BaseHuman {
    constructor(config) {
        super(config)

        // const
        // Store age in the age modifier instead
        this.bodyZones = ['l-eye', 'r-eye', 'jaw', 'nose', 'mouth', 'head',
            'neck', 'torso', 'hip', 'pelvis', 'r-upperarm', 'l-upperarm',
            'r-lowerarm', 'l-lowerarm', 'l-hand', 'r-hand', 'r-upperleg',
            'l-upperleg', 'r-lowerleg', 'l-lowerleg', 'l-foot', 'r-foot', 'ear'
        ]

        // flags
        // this.hasWarpTargets = false // not used yet


        // a modular container for modifiers such as age, left-arm length etc
        this.modifiers = new Modifiers(this)

        // a modular object with human factors and their getters and setters,
        //  e.g. age, weight, ageInYears
        this.factors = new Factors(this)

        // holds loaded targets, target metadata, and target related methods
        this.targets = new Targets(this)

        this.proxies = new Proxies(this)
        this.add(this.proxies)

        this.ethnicSkinBlender = new EthnicSkinBlender(this)

        this.io = new HumanIO(this)

        // because three.js/core/object3d.js sets these in the constructor, preventing method override
        this.onBeforeRender = Human.prototype.onBeforeRender
        this.onAfterRender = Human.prototype.onAfterRender
```

```
    }


    updateHeight() {
        // TODO update position, by reading bone world position, or buffer geom?
        // let position = this.mesh.geometry._bufferGeometry.attributes.position.array
        // let ys=[]
        // for (let i=0;i<position.length;i+=3){
        //     ys.push(position[i])
        // }
        // let miny = _.min(ys)
        // this.position.y=miny
        //
        // Use joint ground, get face group that corresponds, then get vertices for the faces, then get mean y?
        // no the ground join is just between the feet, it doesn't seem to help us
    }

    updateSkinColor() {
-       const defaultSkin = this._skinCache["data/skins/young_caucasian_female/young_caucasian_female.json"]
-       if (defaultSkin) {
-           return defaultSkin.color = this.ethnicSkinBlender.valueOf()
-       } else {
+       if (!this.mesh || !this.mesh.material) {
            return false
        }
+
+       const materials = Array.isArray(this.mesh.material)
+           ? this.mesh.material
+           : (this.mesh.material.materials || [this.mesh.material])
+       const skinMaterial = materials[0]
+
+       if (!skinMaterial || !skinMaterial.color) {
+           return false
+       }
+
+       skinMaterial.color.copy(this.ethnicSkinBlender.valueOf())
+       skinMaterial.needsUpdate = true
+       return true
    }

    /** Call when vertices/element change **/
    onElementsNeedUpdate() {
        super.onElementsNeedUpdate()
+       syncLegacyGeometry(this.mesh.geometry)
        this.updateJointPositions()
        this.proxies.onElementsNeedUpdate()
        this.updateSkinColor()
-       this.mesh.geometry.computeVertexNormals()
    }

    /** Call before render **/
    onBeforeRender() {
        super.onBeforeRender()
        TWEEN.update()
        this.targets.applyTargets()
+       this.updateSkinColor()
        if (this.mesh && this.mesh.geometry.elementsNeedUpdate) {
            this.onElementsNeedUpdate()
        }
    }

    /**
     * This should be called after render
     */
    onAfterRender() {
        super.onAfterRender()
    }

    exportConfig() {
```

```
            // TODO, no need to export modifiers with default values or skin etc
            const json = super.exportConfig()
            json.proxies = this.proxies.exportConfig()
            json.modifiers = this.modifiers.exportConfig()
            return json
        }

        importConfig(json) {
            // json = _.defaults(json, { modifiers: [], proxies: [] })
            super.importConfig(json)
            if (json.proxies)
                this.proxies.importConfig(json.proxies)
            if (json.modifiers)
                this.modifiers.importConfig(json.modifiers)
        }
    }


    // export var human = new Human();
    export default Human
```

FILE: webpack.config.js

Status: MODIFIED | +41 / -77

```
-
-
    const path = require('path')
-   const webpack = require('webpack')

-   const BUILD_DEV = !(process.env.NODE_ENV === "production")
    const OUTPUTDIR = 'dist'
    const SRC_PATH = path.join(__dirname, './src')

-   console.log(`Running in BUILD_DEV=${BUILD_DEV} mode to OUTPUTDIR=${OUTPUTDIR}`)
+   module.exports = (env, argv = {}) => {
+       const isProduction = argv.mode === 'production'
+       const BUILD_DEV = !isProduction

-   // ///////////
-   // Plugins //
-   // ///////////
+       console.log(`Running in BUILD_DEV=${BUILD_DEV} mode to OUTPUTDIR=${OUTPUTDIR}`)

-   // uglify in production
-   const uglifyJsPlugin = new webpack.optimize.UglifyJsPlugin({
-       minimize: !BUILD_DEV,
-       sourceMap: BUILD_DEV,
-       mangle: false,
-       output: {
-       },
-       compress: {
-           // drop_debugger: true,
-           drop_console: !BUILD_DEV,
-           // warnings: false,
-           unused: true,
-           dead_code: true
-       }
-   })
-
-
-   const plugins = [
-   ]
-   if (BUILD_DEV) {
-       // DEVELOPMENT
-       plugins.push(
-       )
-   } else {
```

```
-     console.warn('!!!!! RELEASE BUILD !!!!!\n')
-     plugins.push(
-         // Disable due to issue with this plugin
-         // See https://github.com/webpack/webpack/issues/2644
-         // new webpack.optimize.DedupePlugin(),
-         new webpack.optimize.OccurrenceOrderPlugin(),
-         new webpack.optimize.AggressiveMergingPlugin(),
-         uglifyJsPlugin,
-         new webpack.NoErrorsPlugin()
-     )
- }
-
- // main config
- module.exports = {
-     entry: {makehuman:'./src/bundle.js'},
-     output: {
-         path: path.join(__dirname, OUTPUTDIR),
-         filename: BUILD_DEV ? '[name].js' : '[name].min.js',
-         libraryTarget: 'umd',
-         library: '[name]'
-     },
-     module: {
-         loaders: [
-             {
-                 test: /\.js$/i,
-                 // loaders: ['react-hot',{loader:'babel', query:{cacheDirectory: true}}],
-                 // Note: you need a the .babelrc file to solve
  http://stackoverflow.com/questions/32211649/debugging-with-webpack-es6-and-babel
-                 loader: 'babel',
-                 exclude: /(node_modules|bower_components)/,
-                 query: {
-                     cacheDirectory: 'node_modules/.cache'
-                 }
+     return {
+         entry: { makehuman: './src/bundle.js' },
+         output: {
+             path: path.join(__dirname, OUTPUTDIR),
+             filename: BUILD_DEV ? '[name].js' : '[name].min.js',
+             chunkFilename: BUILD_DEV ? '[name].js' : '[name].min.js',
+             library: {
+                 name: '[name]',
+                 type: 'umd'
              },
-             // this loads it as javascript in one go
-             {
-                 test: /\.json$/,
-                 loader: 'json-loader'
-             },
-         ]
-     },
-     resolve: {
+             clean: true
+         },
+         module: {
+             rules: [
+                 {
+                     test: /\.js$/i,
+                     exclude: /(node_modules|bower_components)/,
+                     use: {
+                         loader: 'babel-loader',
+                         options: {
+                             cacheDirectory: true
+                         }
+                     }
+                 }
+             ]
+         },
+         resolve: {
             modules: [
                 SRC_PATH,
```

```
                './src/js',
                './test',
                './test/mocha',
-               "node_modules",
+               'node_modules'
            ],
-           // extentions to auto add if needed
-           extensions: ["", ".js"]
+           alias: {
+               'three-stdlib-geometry': path.join(__dirname, 'node_modules/three-stdlib/deprecated/Geometry.js')
+           },
+           extensions: ['.js']
        },
-       devtool: BUILD_DEV ? 'source-map' : 'cheap-module-source-map', //  slower than 'cheap-module-eval-source-map'
-       plugins
+           devtool: 'source-map',
+           optimization: {
+               minimize: isProduction,
+               splitChunks: false
+           }
+       }
    }
```

FILE: package.json

Status: MODIFIED | +35 / -52

```
    {
-     "name": "makehuman-js",
+     "name": "character-creator",
      "version": "0.1.2",
-     "description": "Builds 3D human characters in the browser",
+     "description": "Character Creator fork maintained by Ralf Krümmel",
      "main": "src/bundle.js",
-     "repository": "makehuman-js/makehuman-js",
-     "author": "wassname",
-     "license": "AGPLv3",
+     "repository": {
+       "type": "git",
+       "url": "https://github.com/kruemmel-python/Character-Creator.git"
+     },
+     "homepage": "https://github.com/kruemmel-python/Character-Creator",
+     "author": "Ralf Krümmel (https://github.com/kruemmel-python)",
+     "license": "AGPL-3.0-or-later",
      "keywords": [
        "3d",
        "human",
        "model",
        "characters",
        "makehuman"
      ],
      "scripts": {
-       "start": "./node_modules/.bin/webpack-dev-server --inline --progress",
-       "start_windows": ".\\node_modules\\.bin\\webpack-dev-server --inline --progress",
-       "try_release": "NODE_ENV=production npm start",
-       "watch": "webpack --progress --watch",
-       "build": "webpack --progress",
-       "build:debug": "webpack --show-errors",
-       "release": "NODE_ENV=production webpack --progress",
-       "release_windows": "NODE_ENV=production webpack --progress",
-       "test": "./node_modules/.bin/webpack-dev-server --config webpack.test.config.js --open",
-       "test_windows": ".\\node_modules\\.bin\\webpack-dev-server --config webpack.test.config.js --open"
+       "start": "webpack serve --mode development --progress",
+       "start_windows": "webpack serve --mode development --progress",
+       "try_release": "webpack --mode production --progress",
+       "watch": "webpack --mode development --progress --watch",
+       "build": "webpack --mode development --progress",
+       "build:debug": "webpack --mode development --stats-error-details",
```

```
+     "release": "webpack --mode production --progress",
+     "release_windows": "webpack --mode production --progress",
+     "test": "webpack serve --mode development --config webpack.test.config.js --open",
+     "test_windows": "webpack serve --mode development --config webpack.test.config.js --open"
    },
    "dependencies": {
      "d3-random": "^0.2.1",
-     "lodash": "^4.5.1",
-     "qs": "^6.3.0",
-     "three": "^0.82.1",
-     "tween": "^0.9.0",
-     "makehuman-data": "0.0.2"
+     "lodash": "^4.17.21",
+     "makehuman-data": "0.0.2",
+     "qs": "^6.14.0",
+     "three": "0.182.0",
+     "three-stdlib": "^2.36.1",
+     "tween": "^0.9.0"
    },
    "devDependencies": {
-     "babel-cli": "^6.4.5",
-     "babel-core": "^6.18.2",
-     "babel-eslint": "^7.1.0",
-     "babel-loader": "^6.2.7",
-     "babel-plugin-transform-decorators-legacy": "^1.3.4",
-     "babel-plugin-transform-es2015-modules-commonjs-simple": "^6.7.4",
-     "babel-plugin-transform-flow-strip-types": "^6.18.0",
-     "babel-plugin-transform-runtime": "^6.15.0",
-     "babel-polyfill": "^6.16.0",
-     "babel-preset-es2015": "^6.18.0",
-     "babel-preset-es2015-native-modules": "^6.9.4",
-     "babel-preset-es2015-webpack": "^6.4.3",
-     "babel-preset-stage-1": "^6.16.0",
-     "babel-register": "^6.4.3",
-     "babel-runtime": "^6.3.19",
+     "@babel/core": "^7.28.3",
+     "@babel/preset-env": "^7.28.3",
+     "babel-loader": "^10.0.0",
      "bignumber": "^1.1.0",
      "bignumber.js": "^2.3.0",
-     "chai": "^3.5.0",
-     "chai-as-promised": "^5.2.0",
+     "chai": "^4.5.0",
+     "chai-as-promised": "^7.1.2",
      "chai-bignumber": "^1.0.0",
      "chai-things": "^0.2.0",
-     "copy-webpack-plugin": "^4.5.2",
-     "eslint": "^3.10.1",
-     "eslint-config-airbnb": "^13.0.0",
-     "eslint-import-resolver-webpack": "^0.7.0",
-     "eslint-loader": "^1.6.1",
-     "eslint-plugin-import": "^2.2.0",
-     "exports-loader": "^0.6.3",
-     "file-loader": "^0.9.0",
-     "imports-loader": "^0.6.5",
-     "json-loader": "^0.5.4",
+     "copy-webpack-plugin": "^13.0.1",
      "jstat": "^1.5.3",
-     "mocha": "^2.4.5",
-     "raw-loader": "^0.5.1",
-     "url-loader": "^0.5.7",
-     "webpack": "2.1.0-beta.21",
-     "webpack-dev-server": "2.1.0-beta.10"
+     "mocha": "11.3.0",
+     "webpack": "^5.105.0",
+     "webpack-cli": "^6.0.1",
+     "webpack-dev-server": "^5.2.3"
    }
  }
```

FILE: .eslintrc.js

Status: DELETED | +0 / -81

```
- module.exports = {
-     "parser": 'babel-eslint',
-     plugins: [
-     ],
-     extends: [
-         'airbnb'
-     ],
-     "env": {
-         "es6": true,
-         "node": true,
-         "browser": true,
-         // "amd": true,
-         "mocha": true
-     },
-     settings:{
-          'import/resolver': 'webpack'
-     },
-     parserOptions: {
-         ecmaVersion: 6,
-         ecmaFeatures: {
-             experimentalObjectRestSpread: true
-         },
-         sourceType: 'module'
-     },
-     "rules": {
-         "max-len": [1, 180, 2, {"ignoreComments": true}],
-         "quote-props": [1, "consistent-as-needed"],
-         "comma-dangle": [0, "never"],
-         // "no-unused-vars": [1, {"vars": "local", "args": "none"}],
-         "indent": ["error", 4],
-         "no-underscore-dangle": [0],
-         "linebreak-style": ["error", "unix"],
-         "semi": ["warn", "never"],
-         "no-console": 0,
-         "no-use-before-define": 1,
-         "no-else-return": 0,
-         "quotes": 0,
-         'no-mixed-operators': ["error", {"allowSamePrecedence": true}],
-         "import/no-extraneous-dependencies": 'off',
-         "import/extensions": 'off',
-         "import/no-unresolved": 'warn',
-         "no-unused-vars": 1,
-         "no-param-reassign": 0,
-         "curly": 1,
-         "class-methods-use-this": 0,
-         "no-return-assign": 0,
-         "object-curly-spacing": 1,
-         "no-mixed-operators": 0,
-         "no-plusplus": 0,
-         "arrow-body-style": 0
-         // "arrow-parens": 0,
-         // 'arrow-spacing': 'error',
-         // 'block-spacing': 'error',
-         // 'comma-dangle': 'off',
-         // 'comma-spacing': 'error',
-         // 'comma-style': 'error',
-         // 'curly': 'error',
-         // 'dot-notation': 'error',
-         // 'eqeqeq': 'error',
-         // 'eol-last': 'error',
-         // 'indent': ['error', 4, {SwitchCase: 1}],
-         // 'key-spacing': 'error',
-         // 'keyword-spacing': 'error',
-         // 'linebreak-style': ['error', 'unix'],
-         // 'no-console': 'off',
```

```
-         // 'no-param-reassign': 'error',
-         // 'no-tabs': 'error',
-         // 'no-trailing-spaces': 'error',
-         // 'no-underscore-dangle': 'error',
-         // 'no-whitespace-before-property': 'error',
-         // 'quotes': ['error', 'single'],
-         // 'semi': ['error', 'always'],
-         // 'semi-spacing': 'error',
-         // 'space-before-blocks': 'error',
-         // 'space-before-function-paren': ['error', 'never'],
-         // 'space-in-parens': 'error',
-         //
-         // 'react/display-name': 'off',
-         // 'react/prop-types': 'off'
-     }
- }
```

FILE: webpack.test.config.js

Status: MODIFIED | +31 / -36

```
   const path = require('path')
- const webpack = require('webpack')
   const CopyWebpackPlugin = require('copy-webpack-plugin')

- const BUILD_DEV = !(process.env.NODE_ENV === "production")
   const OUTPUTDIR = 'build'
- const SRC_PATH = path.join(__dirname, './src')
-
- console.log(`Running in BUILD_DEV=${BUILD_DEV} mode to OUTPUTDIR=${OUTPUTDIR}`)
-
- // ////////////
- // Plugins //
- // ////////////
-
- // main config
   module.exports = {
-     entry: {test:'./test/index.js'},
+     entry: { test: './test/index.js' },
       output: {
           path: path.join(__dirname, OUTPUTDIR),
-         filename: 'makehuman.test.js'
+         filename: 'makehuman.test.js',
+         clean: true
       },
       module: {
-         loaders: [
+         rules: [
              {
                  test: /\.js$/i,
-                 // loaders: ['react-hot',{loader:'babel', query:{cacheDirectory: true}}],
-                 // Note: you need a the .babelrc file to solve
   http://stackoverflow.com/questions/32211649/debugging-with-webpack-es6-and-babel
-                 loader: 'babel',
                  exclude: /(node_modules|bower_components)/,
-                 query: {
-                     cacheDirectory: 'node_modules/.cache'
+                 use: {
+                     loader: 'babel-loader',
+                     options: {
+                         cacheDirectory: true
+                     }
                  }
-             },
-             // this loads it as javascript in one go
-             {
-                 test: /\.json$/,
-                 loader: 'json-loader'
```

```
-             },
+             }
          ]
      },
      resolve: {
-         // modules: [
-         //     SRC_PATH,
-         //     './src/js',
-         //     './test',
-         //     './test/mocha',
-         //     "node_modules",
-         // ],
-         // extentions to auto add if needed
-         extensions: ["", ".js"]
+         alias: {
+             'three-stdlib-geometry': path.join(__dirname, 'node_modules/three-stdlib/deprecated/Geometry.js')
+         },
+         extensions: ['.js']
      },
      devtool: 'inline-source-map',
-     plugins: [new CopyWebpackPlugin([{ from: 'test/index.html', to:'.' }])],
-     devServer: { port:'8081'}
+     plugins: [
+         new CopyWebpackPlugin({
+             patterns: [{ from: 'test/index.html', to: '.' }]
+         })
+     ],
+     devServer: {
+         port: 8081,
+         static: [
+             {
+                 directory: path.join(__dirname, OUTPUTDIR)
+             },
+             {
+                 directory: path.join(__dirname, 'node_modules/makehuman-data/public'),
+                 publicPath: '/node_modules/makehuman-data/public'
+             }
+         ]
+     }
  }
```

FILE: README.md

Status: MODIFIED | +28 / -28

```
- # makehuman-js
+ # Character Creator

- A library that builds human models in the browser.

- A live demo (NSFW while loading) is available at [mhwebui.wassname.com](http://mhwebui.wassname.com/)
+ https://github.com/user-attachments/assets/24d159c8-bf3e-4b74-926d-d53554bfdff9

- An an example of how to use this library is in the [example
  repository](https://github.com/makehuman-js/makehuman-js-example).

- ![](docs/images/ui-screenshot.png)
+ Fork-Maintainer: **Ralf Kruemmel**
+ GitHub: https://github.com/kruemmel-python/Character-Creator

- # Status
+ Dieses Projekt ist ein Character-Creator auf Basis von `makehuman-js` und `makehuman-data`.

- This is a alpha product.
+ ## Schnellstart

- # Usage
+ 1. `npm install`
```

```
+ 2. `npm start`
+ 3. Browser: `http://localhost:8080/` (oder angezeigter Port)

- An example of how to use the library is available at
  [makehuman-js/makehuman-js-example](https://github.com/makehuman-js/makehuman-js-example).
+ Eine erweiterte Schritt-fuer-Schritt-Anleitung steht in `Installation.md`.

- # Tests
+ ## Features im aktuellen Fork

- - clone
- - `npm install`
- - `npm test` (or on windows use `npm run test_windows`)
- - this will launch a browser which will run mocha unit tests
+ - Human-Rendering im Browser (Three.js)
+ - Modifier-Slider
+ - Haar-Slider
+ - Kleidungs-Slider
+ - T-Pose Button
+ - Export als OBJ und FBX

- # Questions
+ ## Attribution

- Please log issues on github and ask questions on stackoverflow. Otherwise questions can be sent to "makehuman.js at
  wassname dot org".
+ - Dieses Projekt basiert auf `makehuman-js` (Originalprojekt) und Daten aus `makehuman-data`.
+ - Vorhandene Upstream-Urheber- und Lizenzhinweise in den Quelldateien bleiben erhalten.

- # Contributing
+ ## Lizenz

- We welcome contributions as pull request or issues reporting issues you might come across. All contributors of code
  must agree to the licensing terms.
+ Lizenz fuer diesen Fork-Code: **AGPL-3.0-or-later**.
+ Die Daten/Modelle unter `makehuman-data` bleiben unter deren eigener Lizenz.
+ Volltext der Lizenz: `LICENSE`.

- # Acknowledgement

- This browser software is inspired by the desktop python software makehuman.org. It also depends on a makehuman-data
  package which provides data from makehuman. Thanks to major contributors to makehuman who can be found
  [here](http://www.makehuman.org/halloffame.php) and all the minor contributions.
+ https://github.com/user-attachments/assets/de13f133-6003-4c4c-8fa6-ffa0ace7d3df

- # Licensing

- (This license is for the makehuman-js code, outputs are CC0, the data/models remain under the makehuman project
  licence)

- Copyright 2016-2017 Mike Clark (wassname)
+ Weitere Lizenz- und Attribution-Hinweise:

- It is open source and free to use as it is licensed under the AGPLv3.
-
- Alternative commercial license terms are available from if you wish to redistribute it as part of a proprietary
  closed source product or deliver software software-as-a-service (SaaS) using it as part of a proprietary closed
  source service.
-
- Projects or startups in the first two years or making under $2000/year, get a free commercial license. Just message
  me and let me know your using it.
+ - `NOTICE.md`
+ - `COPYRIGHT.md`
```

FILE: src/js/human/proxy.js

Status: MODIFIED | +36 / -13

```
    /**
     * @name            MakeHuman
     * @copyright       MakeHuman Team 2001-2016
     * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     * @description     Manages meshes such as clothes which attatch to the human
     * and it's skeleton
     */

    import _ from 'lodash'
    import * as THREE from 'three'
+   import {
+       createMaterialArray,
+       extractUrlBase,
+       legacyToBufferGeometry,
+       parseLegacyModel,
+       syncLegacyGeometry
+   } from '../helpers/legacy-three-compat'

    /**
     * parseProxyUrl
     * @param  {String} url - e.g. "data/proxies/clothes/Bikini/Bikini.json#blue"
     * @return {Object}     - e.g. {group:"clothes",name:"Bikini",file:"bikini.json", materialName:"blue"}
     */
    export const parseProxyUrl = (address) => {
        // if (!address.includes('#')) address += '#'
        const [fullUrl, materialName] = address.split('#')
        const [, , group, name, file] = fullUrl.match(/(.+\/)*(.+)\/(.+)\/(.+)\.json/)
        const key = `${group}/${name}/${file}.json${materialName ? `#${materialName}` : ''}`
        const thumbnail = `${group}/${name}/${materialName || file}.thumb.png`
        return { group, name, key, materialName: materialName || '', thumbnail }
    }

    export class Proxy extends THREE.Object3D {
        constructor(url, human, manager = new THREE.LoadingManager()) {
            super()

            this.manager = manager
            this.human = human

            // after fitting we expand the size by this fraction
            this.extraGeometryScaling = [1.0, 1.0, 1.0]

-           this.loader = new THREE.XHRLoader(this.manager)
+           this.loader = new THREE.FileLoader(this.manager)

            const { name, group, materialName, key, thumbnail } = parseProxyUrl(url)
            this.key = key
            this.name = name
            this.group = group
            this.thumbnail = thumbnail
            this.materialName = materialName
            this.url = `${this.human.config.baseUrl}proxies/${this.key}`

            this.mesh = null
            this.visible = false
            this.metadata = {}
        }

        /** load a proxy from threejs json file, making it a child object and giving it the same skeleton **/
        load() {
            const self = this
            if (this.mesh) return Promise.resolve(this.mesh)
            return new Promise((resolve, reject) => {
                try {
```

```
                    self.loader.load(self.url,
                            resolve,
                            undefined,
                            reject
                        )
                } catch (e) {
                    reject(e)
                }
            })
            .catch((err) => {
                console.error('Failed to load proxy data', self.url, err)
            })
            .then(text => JSON.parse(text))
            .then((json) => {
                self.metadata = json.metadata
                const texturePath = self.texturePath &&
                    (typeof self.texturePath === 'string') ?
                        self.texturePath :
-                           THREE.Loader.prototype.extractUrlBase(self.url)
-               return new THREE.JSONLoader().parse(json, texturePath)
+                           extractUrlBase(self.url)
+               return parseLegacyModel(json, texturePath)
            })
            // use unpacking here to turn one args into two, as promises only return one
            .then(({
-               geometry,
+               legacyGeometry,
                materials
            }) => {
-               geometry.name = self.url
+               legacyGeometry.name = self.url
                materials.map(m => (m.skinning = true))

-               const mesh = new THREE.SkinnedMesh(geometry, new THREE.MultiMaterial(materials))
+               const mesh = new THREE.SkinnedMesh(
+                   legacyToBufferGeometry(legacyGeometry),
+                   createMaterialArray(materials)
+               )

                // TODO check they are the same skeletons
-               mesh.children.pop() // pop existing skeleton
+               if (mesh.children.length > 0) {
+                   mesh.children.pop() // pop existing skeleton
+               }
                mesh.skeleton = self.human.skeleton

                mesh.castShadow = true
                mesh.receiveShadow = true

                self.mesh = mesh
-               self.mesh.geometry.computeVertexNormals()
+               syncLegacyGeometry(self.mesh.geometry)
                self.add(mesh)

                // when it overlaps with body, show proxy (body has 0, we want higher ints)
                mesh.renderOrder = self.metadata.z_depth

                self.updatePositions()

                // change it to use the material specified in the url hash
                if (self.materialName) {
                    const materialIndex = _.findIndex(materials, material => material.name === self.materialName)
                    self.changeMaterial(materialIndex)
                }

                return mesh
            })
    }
```

```
        /** Turn mesh on or off, loading if needed **/
        toggle(state) {
            if (state === undefined) state = !this.visible
            if (this.visible === state) return Promise.resolve(this)
            this.visible = state
            let promisedMesh
            if (state === false) promisedMesh = Promise.resolve()
            else promisedMesh = this.load().then(() => this.updatePositions())
            return promisedMesh.then(() => this.human.proxies.updateFaceMask())
        }

        /**
         * This recalculates the coords of the proxy using the vertice inds, weights, and offsets
         * like in makehumans's proxy.py:Proxy.getCoords()
         */
        updatePositions() {
            // TODO faster to do this in the gpu
            // equation = vertice = w0 * v0 + w1 * v1 + w2*v2 + offset, where w0 = weights[i][0], v0 = ref_verts_i[0]
            if (!this.visible || !this.mesh) return null
            const o = this.metadata.offsets
            const w = this.metadata.weights
            const v = this.metadata.ref_vIdxs
                .map(row =>
                    row.map(vIndx => this.human.mesh.geometry.vertices[vIndx])
                )

            // convert this.matrix to Matrix3
            const mw = this.matrix.elements
-           const matrix = new THREE.Matrix3()
-           matrix.set(mw[0], mw[1], mw[2], mw[4], mw[5], mw[6], mw[8], mw[9], mw[10]).transpose()
-           const m = matrix.elements
+           let m
+           if (mw.length === 9) {
+               // Some tests inject a Matrix3 directly into this.matrix.
+               m = mw
+           } else {
+               const matrix = new THREE.Matrix3()
+               matrix.set(mw[0], mw[1], mw[2], mw[4], mw[5], mw[6], mw[8], mw[9], mw[10]).transpose()
+               m = matrix.elements
+           }
+           const scaling = new THREE.Vector3(...this.extraGeometryScaling)

            for (let i = 0; i < this.mesh.geometry.vertices.length; i++) {
                // xyz offsets calculated as dot(matrix, offsets)
                const vertice = new THREE.Vector3(
                    o[i][0] * m[0] + o[i][1] * m[1] + o[i][2] * m[2],
                    o[i][0] * m[3] + o[i][1] * m[4] + o[i][2] * m[5],
                    o[i][0] * m[6] + o[i][1] * m[7] + o[i][2] * m[8]
                )

                // Three weights to three vectors
                for (let j = 0; j < 3; j++) {
                    vertice.x += w[i][j] * v[i][j].x
                    vertice.y += w[i][j] * v[i][j].y
                    vertice.z += w[i][j] * v[i][j].z
                }
+               vertice.multiply(scaling)
                this.mesh.geometry.vertices[i] = vertice
            }
-           this.mesh.geometry.scale(...this.extraGeometryScaling)
            this.mesh.geometry.verticesNeedUpdate = true
            this.mesh.geometry.elementsNeedUpdate = true
            return this.mesh.geometry.vertices
        }

        changeMaterial(i) {
            if (i > this.mesh.material.materials.length) return this.mesh.material.materials.length
            this.mesh.geometry.faces.map(face => (face.materialIndex = i))
            this.mesh.geometry.groupsNeedUpdate = true
```

```
        return true
    }

    preRender() {

    }
    onAfterRender() {}
}


/**
 * Container for proxies
 */
export class Proxies extends THREE.Object3D {
    constructor(human) {
        super()
        this.human = human
        // Proxies
        // TODO replace with a better system
        // this._hairProxy = undefined
        // this._eyesProxy = undefined
        // this._eyebrowsProxy = undefined
        // this._eyelashesProxy = undefined
        // this._teethProxy = undefined
        // this._tongueProxy = undefined
        // this._clothesProxies = {}

        this._cache = {}

        // init an object for each proxy but don't load untill needed
        this.human.config.proxies
            .map(url => new Proxy(`${this.human.config.baseUrl}proxies/${url}`, this.human))
            .map(proxy => this.add(proxy))
    }

    /**
     * Toggles or sets a proxy
     * params:
     *   key {String} the url for the proxy  (relative to baseUrl) e.g. eyes/Low-Poly/Low-Poly.json#brown
     *   state {Boolean|undefined} set the proxy on or off or if undefined toggle it
     * returns a promise to load the mesh
     */
    toggleProxy(key, state) {
        // try to find an existing proxy with this key
        let proxy = _.find(this.human.proxies.children, p => p.url === key) ||
            _.find(this.human.proxies.children, p => p.key === key) ||
            _.find(this.human.proxies.children, p => p.name === key)
        // or init a new one
        if (!proxy) {
            console.warn(`Could not find proxy with key ${key}`)
            // throw new Error(`Could not find loaded proxy to toggle with key: ${key}`)
            proxy = new Proxy(`${this.human.config.baseUrl}proxies/${key}`, this.human)
            this.add(proxy)
        } else {
            console.log('toggleProxy', proxy.url, state)
        }
        return proxy.toggle(state)
    }

    updatePositions() {
        this.children.forEach(child => child.updatePositions())
    }

    /** Make faces to hide parts under clothes, see makehuman/plugins/3_libraries_clothes_choose.py:updateFaceMasks
**/
    updateFaceMask(minMaskedVertsPerFace = 3) {
        // get deleted vertices from all active proxies
        const deleteVerts = this.children
            .filter(proxy =>
                proxy.visible &&
```

```
                    proxy.mesh &&
                    proxy.metadata.deleteVerts &&
                    proxy.metadata.deleteVerts.length
                )
                .map(proxy => proxy.metadata.deleteVerts)
            const nbVertices = this.human.mesh.geometry.vertices.length
            const nullMaterial = this.human.mesh.material.materials.findIndex(m => m.name === 'maskedFaces')

            // for each vertice, see if any proxy wants to delete it
            const dv = Array(nbVertices)
            for (let i = 0; i < dv.length; i++) {
                dv[i] = _.sum(deleteVerts.map(vs => vs[i])) > 0
            }

            // if more than n vertices of a face are masked, mask the face else unmask
            this.human.mesh.geometry.faces.map((face) => {
                if ((dv[face.a] + dv[face.b] + dv[face.c]) >= minMaskedVertsPerFace) {
                    face.materialIndex = nullMaterial
                } else {
                    face.materialIndex = face.oldMaterialIndex
                }
                return face.materialIndex
            })
            this.human.mesh.geometry.groupsNeedUpdate = true

            const facesMasked = this.human.mesh.geometry.faces
            .filter(face => face.materialIndex === nullMaterial).length
            console.debug('vertices masked ', _.sum(dv), 'faces masked', facesMasked)
        }

        onElementsNeedUpdate() {
            this.updatePositions()
            this.children
            .filter(proxy => proxy.visible && proxy.mesh)
-           .map(proxy => proxy.mesh.geometry.computeVertexNormals())
+           .map((proxy) => {
+               syncLegacyGeometry(proxy.mesh.geometry)
+               return proxy.mesh.geometry
+           })
        }

        exportConfig() {
            return this.children.filter(p => p.visible).map(p => p.url)
        }

        importConfig(config) {
            this.children.map(p => (p.visible = false))
            this.updateFaceMask()
            return config.map(url => this.toggleProxy(url, true))
        }
    }


    export default Proxy
```

FILE: Installation.md

Status: ADDED | +46 / -0

```
+ # Installation und Ausfuehrung
+
+ Diese Anleitung gilt fuer den Fork **Character Creator** von **Ralf Kruemmel**:
+
+ - Repository: `https://github.com/kruemmel-python/Character-Creator`
+
+ ## 1. Voraussetzungen
+
+ - Node.js (empfohlen: aktuelle LTS)
```

+ - npm
+
+ Versionen pruefen:
+
+ ```powershell
+ node -v
+ npm -v
+ ```
+
+ ## 2. Projekt holen
+
+ ```powershell
+ git clone https://github.com/kruemmel-python/Character-Creator.git
+ cd Character-Creator
+ ```
+
+ Falls das Projekt schon lokal vorhanden ist, nur in den Ordner wechseln.
+
+ ## 3. Abhaengigkeiten installieren
+
+ ```powershell
+ npm install
+ ```
+
+ ## 4. Entwicklungsserver starten
+
+ ```powershell
+ npm start
+ ```
+
+ Danach im Browser oeffnen:
+
+ - `http://localhost:8080/`
+ - oder den von webpack angezeigten Port (z. B. `8081`)
+
+ ## 5. Builds
+
+ Entwicklungsbuild:
+
+ ```powershell
+ npm run build
+ ```
+
+ Release-Build:
+
+ ```powershell
+ npm run release
+ ```
+
+ Ausgabe liegt in `dist/`.
+
+ ## 6. Tests
+
+ ```powershell
+ npm test
+ ```
+
+ ## 7. Optionaler Security-Check
+
+ ```powershell
+ npm audit --json
+ ```

FILE: test/mocha/fixtures.js

Status: MODIFIED | +19 / -11

```
    /** not working yet **/
    import {
        Human
    } from '../../src/js/human/human.js';

    import humanConfig from 'makehuman-data/public/data/resources.json'


    import chai from 'chai'
    import chaiBigNumber from 'chai-bignumber'
    import chaiThings from 'chai-things'
    import chaiAsPromised from 'chai-as-promised'
    chai.use(chaiBigNumber)
    chai.use(chaiThings)
    chai.use(chaiAsPromised)
    export const expect = chai.expect
    export const assert = chai.assert
    export const should = chai.should

    humanConfig.baseUrl = '../node_modules/makehuman-data/public/data/'
    export const config = humanConfig

    export function loadHumanWithoutTargets(human) {
        human = new Human(config);
        // load human and check it worked
-       const promise = human.loadModel()
-       // use chai-as-promised to test the returned promise
-       return expect(promise).to.eventually.have.deep.property('mesh.geometry.vertices').then(() => {return human})
+       return human.loadModel()
+           .then((loadedHuman) => {
+               const result = loadedHuman || human
+               if (!result.mesh || !result.mesh.geometry || !result.mesh.geometry.vertices) {
+                   throw new Error('loadHumanWithoutTargets: mesh.geometry.vertices is missing after loadModel().')
+               }
+               return result
+           })
    }

    /**
     * Fixture that loads a human with targets onto the input variable. Returns a promise
     * @param  {[type]} human [description]
     * @return {Promise}       a promise to load human onto the var human
     */
    export function loadHumanWithTargets(human) {
        human = new Human(config)
        // load human and check it worked
-
-       const promise = human.loadModel()
-       .then(() => {
-           return human.loadTargets()
-           .then((targets) => {return human})
-       })
-       // use chai-as-promised to test the returned promise
-       return expect(promise).to.eventually.have.deep.property('targets.children').then(() => human)
+       return human.loadModel()
+           .then((loadedHuman) => {
+               const result = loadedHuman || human
+               return result.loadTargets().then(() => result)
+           })
+           .then((loadedHuman) => {
+               if (!loadedHuman.targets || !loadedHuman.targets.children) {
+                   throw new Error('loadHumanWithTargets: targets.children is missing after loadTargets().')
+               }
+               return loadedHuman
+           })
```

```
}
```

FILE: NOTICE.md

Status: ADDED | +21 / -0

```
+ # NOTICE
+
+ Project: Character Creator
+ Repository: https://github.com/kruemmel-python/Character-Creator
+ Maintainer: Ralf Kruemmel
+
+ This project is a maintained fork of `makehuman-js` and uses data from `makehuman-data`.
+
+ ## Upstream Attribution
+
+ 1. makehuman-js (upstream project)
+ - Upstream repository: https://github.com/makehuman-js/makehuman-js
+ - Original author attribution in upstream/project files includes Mike Clark ("wassname")
+ - Source file headers also reference MakeHuman Team attribution
+ - Upstream code license basis: AGPL v3 family
+
+ 2. makehuman-data (data package)
+ - Package: `makehuman-data@0.0.2`
+ - Package metadata license reference: http://www.makehuman.org/license.php
+ - README states the data package inherits the makehuman.org AGPLv3-derived license
+ - Contributor reference: http://www.makehuman.org/halloffame.php
+
+ 3. Other npm dependencies
+ - Additional third-party libraries are used under their own licenses
+ - See `node_modules/<package>/LICENSE*` and package metadata for details
+
+ ## Fork Notice
+
+ Fork-specific maintenance and integration work is provided in this repository by Ralf Kruemmel.
```

FILE: test/mocha/targets_spec.js

Status: MODIFIED | +7 / -13

```
    import {
        Targets,
        Target,
        targets,
        TargetMetaData
    } from '../../src/js/human/targets.js'
    import targetList from 'makehuman-data/src/json/targets/target-list.json'
    import targetCategories from 'makehuman-data/src/json/targets/target-category-data.json'
    import Human from '../../src/js/human/human.js'
    import _ from 'lodash'

    import { loadHumanWithTargets, expect } from './fixtures.js'


    describe('targets.js', () => {
        let target
        let human
        before(() => {
            return loadHumanWithTargets()
                .then((loadedHuman) => {
                    human = loadedHuman
                    human.mesh.geometry._bufferGeometry = {}
                    const name = Object.keys(human.targets.children)[0]
                    target = human.targets.children[name]
                })
        })
```

```
    beforeEach('reset factors and targets', () => {
        human.modifiers.reset() // reset modifiers
        Object.keys(human.morphTargetDictionary).map((name) => {
            human.morphTargetInfluences[name] = 0
        })
    })

    describe('Target', () => {
        const targetName = "base/test/testdata/data/targets/chin/chin-triangle.target"

        it('shoud construct', () => {
            const target = new Target(targetName)
            expect(target).to.be.a('object')
        })

        describe('properties', () => {
            // macroVariables, variables, categories, path, group, name
            Object.keys(targetList.targets).forEach((targetName) => {
                  it(targetName, () => {
                      const target = human.targets.children[targetName]
                      expect(target.macroVariables).to.be.an("array")
                      expect(target.variables).to.be.an("array")
                      expect(target.categories).to.be.an("object")
                      expect(target.path).to.be.a("string")
                        .that.has.length.above(0)
                      expect(target.group).to.be.a("string")
                        .that.has.length.above(0)
                      expect(target.name).to.be.a("string")
                        .that.has.length.above(0)
                  });
            })
        })

        describe('methods', () => {

            describe('get and set', () => {
                Object.keys(targetList.targets).forEach((targetName) => {
                      describe(targetName, () => {
                          it('should get ', () => {
                              const target = human.targets.children[targetName]
                              const res = target.value
                              expect(res).to.be.a("number")
                              .that.equals(human.morphTargetInfluences[human.morphTargetDictionary[target.name]])
                          })

                          it('should set', () => {
                              const target = human.targets.children[targetName]
                              const v = target.value = 0.694

expect(human.morphTargetInfluences[human.morphTargetDictionary[target.name]]).to.be.closeTo(v, 0.0001)
                          })
                      })
                  })


            })
            //
            // describe('toMorphTarget', () => {
            //
            //
            //     it('should create target vertices', () => {
            //         target.toMorphTarget(human)
            //             // make sure vertices add up
            //         const idealVal = human.mesh.geometry.vertices[119].clone().add({
            //             x: -0.05,
            //             y: 0.001,
            //             z: 0.001
            //         })
            //         expect(target.vertices[119]).to.deep.equal(idealVal)
```

```
            //     })
            //
            //     it('should apply to human', () => {
            //         target.toMorphTarget(human)
            //         human.mesh.updateMorphTargets()
            //         const i = human.morphTargetDictionary[target.name]
            //         const morphTarget = human.mesh.geometry.morphTargets[i]
            //         expect(morphTarget).to.equal(target)
            //     })
            //
            //     //
            //     // it('should apply to only one group', function () {
            //     //     target.toMorphTarget(human,'body')
            //     //     // TODO implements this, then find vertices on a differen't material index
            //     //     // and check they haven't changed
            //     // });
            //
            //     it('should have correct amount of differing vertices', () => {
            //         target.toMorphTarget(human)
            //         human.mesh.updateMorphTargets()
            //         const i = human.morphTargetDictionary[target.name]
            //         const origVerts = human.mesh.geometry.vertices
            //         const morphVerts = human.mesh.geometry.morphTargets[i].vertices
            //
            //         const diffs = []
            //         for (let i = 0; i < origVerts.length; i++) {
            //             if (!origVerts[i].equals(morphVerts[i])) {
            //                 const diff = origVerts[i].clone().sub(morphVerts[i]).length()
            //                 diffs.push(diff)
            //             }
            //         }
            //
            //         expect(_.max(diffs)).to.be.above(0.05)
            //         const inputs = _.keys(target.dVertices).length
            //         expect(diffs.length).to.be.equal(inputs)
            //     })
            // })
        })
    })

    describe('TargetMetaData', () => {
        it('should construct', () => {
            const targets = new TargetMetaData()
            expect(targets).to.be.a('object')
        })

        describe('methods', () => {
            let targets
            beforeEach(() => {
                targets = new TargetMetaData()
            })

            describe('getTargetsByGroup', () => {
                it('should work for example input', () => {
                    const tgroup = targets.getTargetsByGroup('expression-units-nose,compression')
                    expect(tgroup).to.have.length(3)
                        // expect it to be a Target
                    expect(tgroup[0].constructor.name).to.equal("Target")
                })
            })

            describe('pathToGroupAndCategories', () => {
                it('should work for "height old female.."', () => {
                    const res = targets.pathToGroupAndCategories('data/targets/macrodetails/height/female-old-average
muscle-averageweight-minheight.target')
                    expect(res.categories).to.deep.equal({
                        weight: 'averageweight',
                        gender: 'female',
                        age: 'old',
                        height: 'minheight',
```

```
                        breastfirmness: null,
                        breastsize: null,
                        race: null,
                        muscle: 'averagemuscle',
                        bodyproportions: null
                    })
                    expect(res.group).to.equal("macrodetails-height")
                })

                it('should not change a group input', () => {
                    const input = "expression-units-eyebrows-right-extern"
                    const res = targets.pathToGroupAndCategories(input)
                    expect(res.group).to.equal(input)
                })

                // targets.targetUrls.forEach(path=>{
                //     it('should have non overlapping results for '+path, function () {
                //         var res=targets.pathToGroupAndCategories(path)
                //         expect(res.group).to.not.have.members(res.categories);
                //     });
                // })
            })


            describe('findTargets', () => {
                it('should output nested arrays', () => {
                    const res = targets.findTargets('torso-torso-vshape-less')
-                       // should be an array of string array pairs
-                       // e.g. [['a',['b','c',...]],...]
-                   expect(res).to.be.an('array')
-                       .that.has.deep.property('[0]')
-                       .that.is.an('array')
-                       .that.has.deep.property('[0]')
-                       .that.is.a('string')
-                   expect(res)
-                       .to.have.deep.property('[0]')
-                       .that.has.deep.property('[1]')
-                       .that.is.an('array')
-                       .that.has.deep.property('[0]')
-                       .that.is.a('string')
+                   // should be an array of string array pairs
+                   // e.g. [['a',['b','c',...]],...]
+                   expect(res).to.be.an('array').that.is.not.empty
+                   expect(res[0]).to.be.an('array').with.length.above(1)
+                   expect(res[0][0]).to.be.a('string')
+                   expect(res[0][1]).to.be.an('array').that.is.not.empty
+                   expect(res[0][1][0]).to.be.a('string')
                })

                it('should work for predefined inputs', () => {
                    const res = targets.findTargets('eyes-l-eye-corner2-up')
                    expect(res)
                        .to.deep.equal([
                            ['data/targets/eyes/l-eye-corner2-up.target', ['eyes-l-eye-corner2-up']]
                        ])
                })

                it('should return empty', () => {
                    const res = targets.findTargets('asd49gfdjlfds-feg354tgredfsdc32')
                    expect(res)
                        .to.be.empty
                })
            })
        })
    })

    describe('Targets', () => {
        it('should construct', () => {
            const human = new Human()
```

```
            const targets = new Targets(human)
            expect(targets).to.be.a('object')
            expect(targets).to.have.property('human').that.equals(human)
        })


        describe('applyTargets', () => {

            it('should have different vertices after applying', () => {
                const oldVertices = human.mesh.geometry.vertices.map(vert => vert.clone())
                human.modifiers.randomize()
                human.targets.lastBake = 0
                console.log(human.targets.applyTargets())
                const diffsVerts = _.map(human.mesh.geometry.vertices, (vert, i) => vert.distanceTo(oldVertices[i]))
                    .filter(diff => diff !== 0)

                expect(diffsVerts).to.have.length.above(0)
            })

            it('should get the same value after applying', () => {
                human.modifiers.randomize()
                const oldVals = _.map(human.targets.children, target => target.value)
                human.targets.lastBake = 0
                human.targets.applyTargets()

                const diffsVals = _.map(human.targets.children, target => target.value)
                    .map((val, i) => val - oldVals[i])
                    .filter(diff => diff !== 0)

                expect(diffsVals).to.have.length(0)
            })

        })
    })
})
```

FILE: COPYRIGHT.md

Status: ADDED | +15 / -0

```
+ # COPYRIGHT
+
+ ## Fork
+
+ Copyright (c) 2024-2026 Ralf Kruemmel
+ Repository: https://github.com/kruemmel-python/Character-Creator
+
+ This covers fork-specific modifications made in this repository, where applicable.
+
+ ## Upstream Notices (Preserved)
+
+ This codebase contains and derives from upstream work with existing notices, including:
+
+ - MakeHuman Team (notices in source headers, e.g. 2001-2016 references)
+ - Mike Clark ("wassname") notices in upstream project documentation (e.g. 2016-2017 references)
+ - makehuman-data package notices and contributor references from makehuman.org
+
+ These upstream attributions are intentionally preserved and not removed.
+
+ ## License References
+
+ - Fork package license metadata: `AGPL-3.0-or-later` (see `package.json`)
+ - Data package license reference: http://www.makehuman.org/license.php
+
+ For attribution details, see `NOTICE.md`.
```

FILE: src/js/helpers/OBJExporter.js

Status: MODIFIED | +7 / -4

```
    /**
     * from three.js
     */
    import * as THREE from 'three'
    import _ from 'lodash'
+   import { Geometry as LegacyGeometry } from 'three-stdlib-geometry'

    export const OBJExporter = function () {}
    // TODO add metadata
    // TODO add groups
    OBJExporter.prototype = {

        constructor: OBJExporter,

        parse(object) {
            let output = ''
            const precision = 6

            let indexVertex = 0
            let indexVertexUvs = 0
            let indexNormals = 0

            let i,
                j,
                k,
                l,
                m,
                group,
                face = []

            const parseMesh = function (mesh) {
                const nbVertex = 0
                const nbNormals = 0
                const nbVertexUvs = 0

                let geometry = mesh.geometry
                let faceGroups,
                    groupNames

-               if (geometry instanceof THREE.Geometry) {
+               if (geometry instanceof LegacyGeometry || (geometry && geometry.vertices && geometry.faces)) {
                    faceGroups = geometry.faces.map(f => f.materialIndex)
-                   groupNames = mesh.material.materials.map(m => m.name)
+                   const meshMaterials = mesh.material.materials || mesh.material || []
+                   groupNames = meshMaterials.map(m => m.name)

                    // shortcuts
                    const vertices = mesh.geometry.vertices
                    // const normals = geometry.getAttribute('normal')
                    const uvs = mesh.geometry.faceVertexUvs[0]
                    const faces = mesh.geometry.faces

                    // name of the mesh object
                    output += `o ${mesh.name}\n`

                    // name of the mesh material
                    if (mesh.material && mesh.material.name) {
                        output += `usemtl ${mesh.material.name}\n`
                    }

                    // vertices

                    if (vertices !== undefined) {
                        for (let ii = 0; ii < vertices.length; ii++) {
-                           const vertex = vertices[ii]
```

```
+                           const vertex = vertices[ii].clone()

                            // transfrom the vertex to world space
                            vertex.applyMatrix4(mesh.matrixWorld)

                            // transform the vertex to export format
                            output += `v ${_.round(vertex.x, precision)} ${_.round(vertex.y, precision)}
${_.round(vertex.z, precision)}\n`
                        }
                    }

                    // uvs

                    if (uvs !== undefined) {
                        for (let ii = 0; ii < uvs.length; ii++) {
                            for (let jj = 0; jj < uvs[ii].length; jj++) {
                                const uv = uvs[ii][jj]
                                // transform the uv to export format
                                output += `vt ${_.round(uv.x, precision)} ${_.round(uv.y, precision)}\n`
                            }
                        }
                    }

                    // normals

                    // if (normals !== undefined) {
                    //
                    //     normalMatrixWorld.getNormalMatrix(mesh.matrixWorld);
                    //
                    //     for (i = 0, l = normals.count; i < l; i++, nbNormals++) {
                    //
                    //         normal.x = normals.getX(i);
                    //         normal.y = normals.getY(i);
                    //         normal.z = normals.getZ(i);
                    //
                    //         // transfrom the normal to world space
                    //         normal.applyMatrix3(normalMatrixWorld);
                    //
                    //         // transform the normal to export format
                    //         output += 'vn ' + _.round(normal.x, precision) + ' ' + _.round(normal.y, precision) + ' '
+ _.round(normal.z, precision) + '\n';
                    //
                    //     }
                    //
                    // }

                    for (let ii = 0; ii < faces.length; ii++) {
                        face = faces[ii]

                        // convert from materialIndex to facegroup
                        if (faceGroups && faceGroups[ii] !== group) {
                            group = faceGroups[ii]
                            output += `g ${groupNames[group]}\n`
                        }

                        // transform the face to export format
                        output += `f ${face.a + 1}/${ii * 3 + 1} ${face.b + 1}/${ii * 3 + 2} ${face.c + 1}/${ii * 3 +
3}\n`
                    }
                } else {
                    console.warn('THREE.OBJExporter.parseMesh(): geometry type unsupported', geometry)
                }

                // update index
                indexVertex += nbVertex
                indexVertexUvs += nbVertexUvs
                indexNormals += nbNormals
            }

            const parseLine = function (line) {
```

```
            let nbVertex = 0

            let geometry = line.geometry
            const type = line.type

-           if (geometry instanceof THREE.Geometry) {
+           if (geometry instanceof LegacyGeometry || (geometry && geometry.vertices && geometry.faces)) {
                geometry = new THREE.BufferGeometry().setFromObject(line)
            }

            if (geometry instanceof THREE.BufferGeometry) {
                // shortcuts
                const vertices = geometry.getAttribute('position')
                const indices = geometry.getIndex()

                // name of the line object
                output += `o ${line.name}\n`

                if (vertices !== undefined) {
+                   const vertex = new THREE.Vector3()
                    for (i = 0, l = vertices.count; i < l; i++, nbVertex++) {
                        vertex.x = vertices.getX(i)
                        vertex.y = vertices.getY(i)
                        vertex.z = vertices.getZ(i)

                        // transfrom the vertex to world space
                        vertex.applyMatrix4(line.matrixWorld)

                        // transform the vertex to export format
                        output += `v ${vertex.x} ${vertex.y} ${vertex.z}\n`
                    }
                }

                if (type === 'Line') {
                    output += 'l '

                    for (j = 1, l = vertices.count; j <= l; j++) {
                        output += `${indexVertex + j} `
                    }

                    output += '\n'
                }

                if (type === 'LineSegments') {
                    for (j = 1, k = j + 1, l = vertices.count; j < l; j += 2, k = j + 1) {
                        output += `l ${indexVertex + j} ${indexVertex + k}\n`
                    }
                }
            } else {
                console.warn('THREE.OBJExporter.parseLine(): geometry type unsupported', geometry)
            }

            // update index
            indexVertex += nbVertex
        }

        object.traverse((child) => {
            if (child instanceof THREE.Mesh) {
                parseMesh(child)
            }

            if (child instanceof THREE.Line) {
                parseLine(child)
            }
        })

        return output
    }
```

```
}
export default OBJExporter
```

FILE: test/mocha/ethnic-skin-blender_spec.js

Status: MODIFIED | +2 / -2

```
import _ from 'lodash'
import * as THREE from 'three'

import {
    Human
} from '../../src/js/human/human.js'
import { config, expect } from './fixtures.js'

describe('ethnic-skin-blender.js', () => {
    describe('ethnicSkinBlender', () => {
        it('description', (done) => {
            const human = new Human(config)
            human.loadModel(config)
                .then(() => {
-                   const color = self.human.ethnicSkinBlender.valueOf()
-                   expect(color.isColor).to.be(true)
+                   const color = human.ethnicSkinBlender.valueOf()
+                   expect(color.isColor).to.equal(true)
                })
                .then(() => done())
                .catch(error => done(error))
        })
    })
})
```

FILE: src/js/human/human-modifier.js

Status: MODIFIED | +2 / -1

```
/**
 * @name            MakeHuman
 * @copyright       MakeHuman Team 2001-2016
 * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
 * @author          wassname
+* @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
 * @description
 *
 * Provides classes for modifiers read from makehuman
 *
 * These modifiers are the parameters you slide, and they interact to product
 * a matrix of changes to the targets.
 */

import _ from 'lodash'
import d3Random from 'd3-random'

import {
    targetMetaData
} from './targets'
import modelingModifiers from 'makehuman-data/src/json/modifiers/modeling_modifiers.json'
import measurementModifiers from 'makehuman-data/src/json/modifiers/measurement_modifiers.json'

/**
 * The most basic modifier. All modifiers should inherit from this, directly or
 * indirectly
 *
 * A modifier manages a set of targets applied with a certain weight that
 * influence the human model.
```

```
 */
export class Modifier {
    constructor(groupName, name) {
        this.groupName = groupName
        this.name = name
        this.fullName = `${this.groupName}/${this.name}`

        this.targets = []
        this.description = ""
        this.human = null
        this.defaultValue = 0
        this.min = 0
        this.max = 1

        this.showMacroStats = false

        // Macro variable controlled by this modifier
        this.macroVariable = null
            // Macro variables on which the targets controlled by this modifier depend
        this.macroDependencies = []

        this._symmModifier = null
        this._symmSide = 0

        this.targetMetaData = targetMetaData
    }

    resetValue() {
        const oldVal = this.getValue()
        this.setValue(this.defaultValue)
        return oldVal
    }

    /** Propagate modifier update to dependent modifiers**/
    propagateUpdate(realtime = false) {
        let f
        if (realtime) {
            f = ['macrodetails', 'macrodetails-universal']
        } else {
            f = null
        }

        const modifiersAffectedBy = this.parent.getModifiersAffectedBy(this, f)
            .map((dependentModifierGroup) => {
                // Only updating one modifier in a group should suffice to update the
                // targets affected by the entire group.
                const m = this.parent.getModifiersByGroup(dependentModifierGroup)[0]
                if (realtime) {
                    return m.updateValue(m.getValue(), true)
                } else {
                    return m.setValue(m.getValue(), true)
                }
            })

        return modifiersAffectedBy
    }

    clampValue(value) {
        return _.clamp(value, this.min, this.max)
    }

    /** Subclasses must override this **/
    getFactors(value) {
        throw new Error("NotImplemented")
    }

    /**
     * Gets modifier value from sum of own or given targets
     * @param  {Array} targets=this.targets - The targets to get values from
```

```
 * @return {Number}                    - sum of values from targets
 */
getValue(targets = this.targets) {
    let sum = 0
    for (let i = 0; i < targets.length; i++) {
        const path = targets[i][0]
        const target = this.parent.human.targets.children[path]
        if (!target) {
            // console.error('Target not found for modifier', path, this.name)
            throw new Error(`Target not found for modifier ${path} ${this.name}`)
        } else {
            sum += target.value
        }
    }

    // var targets = _.map(this.targets, target => this.parent.human.targets.children[target[0]])
    // return _.sum(_.map(targets,target=>target.value));
    return sum
}

/**
 * Update the values of this modifers targets
 * @param  {Number} value            new value
 * @param  {Boolean} skipUpdate=false Flag to prevent infinite recursion
 */
updateValue(value, skipUpdate = false) {
    // Update detail state
    if (value !== undefined) {
        this.setValue(value, true)
    }

    // values are directly put into influences matrix now
    // Apply changes
    // for (var i = 0; i < this.targets.length; i++) {
    //     // find the actual target attached to human
    //     let targetName = this.targets[i][0]
    //     let target = this.parent.human.targets.children[targetName]
    //     if (!target) console.warn('Tried to apply modifier but target is not loaded', this.name, targetName)
    //         // have target apply itself to human.mesh
    //     target.value=value
    // }


    if (skipUpdate) {
            // Used for dependency updates (avoid dependency loops && double updates to human)
        return value
    }

    // Update dependent modifiers
    return this.propagateUpdate(true) // realtime=true
}


/**
 * The side this modifier takes in a symmetric pair of two modifiers.
 * Returns 'l' for left, 'r' for right.
 * Returns null if symmetry does not apply to this modifier.
 **/
getSymmetrySide(path = this.name.split('-')) {
    if ('l' in path) {
        return 'l'
    } else if ('r' in path) {
        return 'r'
    } else {
        return null
    }
}

getSymmModifier(path = this.name.split('-')) {
```

```
        return _.map(path, (p) => {
            if (p === 'r') return 'l'
            else if (p === 'l') return 'r'
            else return p
        }).join('-')
    }

    /**
    Get name of the modifier which is symmetric to this one or null if there is none
    **/
    getSymmetricOpposite() {
        if (this._symmModifier) {
            return `${this.groupName}/${this._symmModifier}`
        } else {
            return null
        }
    }

    /**
     * Retrieve the other modifiers of the same type on the human.
     * @return {Array} Array of modifiers with the same class
     */
    getSimilar() {
        // return [m for m in this.parent.getModifiersByType(this.type) if m != self]
        return this.parent.getModifiersByType(this.constructor).filter(m => m !== this)
    }

    isMacro() {
        return this.macroVariable !== null
    }

    get leftLabel() {
        return this.left ? this.left.split('-').slice(-1)[0] : ''
    }
    get rightLabel() {
        return this.right ? this.right.split('-').slice(-1)[0] : ''
    }
    get midLabel() {
        return this.mid ? this.mid.split('-').slice(-1)[0] : ''
    }
    get image() {
        return `data/targets/${this.fullName.replace('/', '/images/').replace('|', '-').toLowerCase()}.png`
    }
}


// class SimpleModifier // Simple modifier constructed from a path to a target file.


/**
 * Modifier that uses the targets module for managing its targets.
 * Abstract baseclass
 */
export class ManagedTargetModifier extends Modifier {

    /**
     * Gets weight for this modifiers targets.
     * @param  {Number} value=1                 The weights sum to this tptal
     * @param  {Array} targets=this.targets     Targets to get weights for
     * @param  {Object} factors=this.getFactors() Name,values for factors and weights
     * @param  {Number} total=1                 The weights sum to this total
     * @return {Object}                         Target paths along with their weights
     *                                             e.g. {data/targets/head/head-age-less.target: -0}
     */
    getTargetWeights(value = 1, targets = this.targets, factors = this.getFactors(value), total = 1) {
        let facVals
        const result = {}

        for (let i = 0; i < targets.length; i++) {
```

```
            const tpath = targets[i][0]
            const tfactors = targets[i][1]

            // look up target factors in our factor values
            facVals = _.map(tfactors, factor => factors[factor] !== undefined ? factors[factor] : 1.0)

            // debug check for unfound factors
            const notFound = _.map(tfactors, factor => factors[factor] === undefined ? factor :
null).filter(_.isString)
            if (notFound.length > 0) {
                console.warn('Names not found in factors', {
                    notFound,
                    modifiersName: this.name,
                    factors
                })
            }



            // debug check for NaN, undefined, null
            if (_.filter(facVals, n => !_.isFinite(n)).length) { console.debug('Some factor values are not finite
numbers', facVals, this.name) }
                // console.debug('factor values',facVals,this.name)

            // so now we multiply the target weight by all modifying factors
            // armlength-old-tall = 1 * 0.10 old * 0.60 tall = 0.6
            result[tpath] = total * _.reduce(facVals, (accum, val) => accum * val, 1)
        }
        return result
    }

    /**
     * Find the groups each child target belongs to
     * @param  {String} path e.g. "data/targets/macrodetails/universal-female-young-maxmuscle-averageweight.target"
     * @return {Array}      e.g. ["age", "gender", "muscle", "weight"]
     */
    findMacroDependencies(path) {
        const result = []
        // get child targets
        const targetPaths = targetMetaData.getTargetsByGroup(path) || []
        for (let i = 0; i < targetPaths.length; i++) {
            const cats = targetPaths[i].macroVariables
            if (cats) result.push(...cats)
        }
        return _.uniq(result)
    }

    /**
     * Set value of this modifier
     * @param {Number} value
     * @param {Boolean} skipDependencies - A flag to avoid infinite recursion
     */
    setValue(value, skipDependencies = false) {
        if (!Number.isFinite(value)) throw new Error(`value is not finite ${value}`)
        value = this.clampValue(value)
        // const factors = this.getFactors(value)
        const tWeights = this.getTargetWeights(value)
        for (const tpath in tWeights) {
            if (tWeights.hasOwnProperty(tpath)) {
                const tWeight = tWeights[tpath]
                const target = this.parent.human.targets.children[tpath]
                if (target === undefined) {
                    if (!this.parent.human.targets.loading) {
                        console.warn('Target not found in', _.keys(this.parent.human.targets.children).length, '
loaded targets. Target=', tpath, '. Modifier=', this.name)
                    }
                } else {
                    target.value = tWeight
                }
            }
        }
```

```
        // console.debug('Set target values',this.name,_.keys(tWeights).length,tWeights)

        if (skipDependencies) {
            return
        }

        // Update dependent modifiers
        this.propagateUpdate(false)
    }

    getValue() {
        // here the right overrides the left
        const right = super.getValue(this.r_targets)
        if (right) { return right } else {
            return -1 * super.getValue(this.l_targets)
        }
    }

    /**
     * Returns weights for each factor e.g {'old':0.8,'young':0.2,child:0}
     */
    getFactors(value) {
        const categoryNames = Object.keys(targetMetaData.targetCategories)
        // return _.map(categoryNames, name => [name, this.parent.human.factors[name + 'Val']]); // returns nested
arrays
        return _.transform(categoryNames, (res, name) => res[name] = this.parent.human.factors[`${name}Val`], {})
    }
}


export class UniversalModifier extends ManagedTargetModifier {
    /**
     * Simple target-based modifier that controls 1, 2 or 3 targets, managed by
     * the targets module.
     * @param  {String} groupName  e.g. head
     * @param  {String} targetName e.g. head-age
     * @param  {String} leftExt    Howtarget relates to modifier
     *                             e.g. less|shorter|narrower|skinnier
     * @param  {String} rightExt   e.g. more|taller|wider|fatter
     * @param  {String} centerExt  e.g. normal
     * @return {undefined}
     */
    constructor(groupName, target, leftExt, rightExt, centerExt) {
        let name,
            targetName
        targetName = `${groupName}-${target}`

        let left = leftExt ? `${targetName}-${leftExt}` : null
        let right = rightExt ? `${targetName}-${rightExt}` : null
        let center = centerExt ? `${targetName}-${centerExt}` : null

        // it either has 3, 2, or 1 targets. Include each target in the name
        if (left && right && center) {
            targetName = `${targetName}-${leftExt}|${centerExt}|${rightExt}`
            name = `${target}-${leftExt}|${centerExt}|${rightExt}`
        } else if (leftExt && rightExt) {
            targetName = `${targetName}-${leftExt}|${rightExt}`
            name = `${target}-${leftExt}|${rightExt}`
        } else {
            right = targetName
            name = target
        }

        super(groupName, name)
        // can't use this before super so we assign to this after
        this.left = left
        this.right = right
        this.center = center
        this.targetName = targetName
```

```
        // console.debug("UniversalModifier(%s, %s, %s, %s)  :  %s", this.groupName, targetName, leftExt, rightExt,
this.fullName)
        this.l_targets = this.targetMetaData.findTargets(this.left)
        this.r_targets = this.targetMetaData.findTargets(this.right)
        this.c_targets = this.targetMetaData.findTargets(this.center)

        this.macroDependencies = _.concat(
            this.findMacroDependencies(this.left),
            this.findMacroDependencies(this.right),
            this.findMacroDependencies(this.center)
        )

        this.targets = _.concat(this.l_targets, this.r_targets, this.c_targets)


        this.min = this.left ? -1 : 0
    }

    /**
     * For a managedTargetModifier we assign the value to the left or right target
     * @param  {Number} value =1 - Number to set modifiers to
     * @return {Object}          - Value for each target and human factor
     *                             e.g. {'african': 0.3333333333333326, // factor
                                        'armslegs-r-lowerarm-fat': 1, // target
                                        'armslegs-r-lowerarm-skinny': -0.0,...}
     */
    getFactors(value = 1) {
        const factors = super.getFactors(value)

        if (this.left !== null) {
            factors[this.left] = -Math.min(value, 0)
        }
        if (this.center !== null) { factors[this.center] = 1.0 - Math.abs(value) }
        factors[this.right] = Math.max(0, value)

        return factors
    }
}

/**
 * Modifiers that control many other modifiers instead of controlling target weights directly
 */
export class MacroModifier extends ManagedTargetModifier {
    constructor(groupName, name) {
        super(groupName, name)
        this.defaultValue = 0.5

        // console.debug("MacroModifier(%s, %s)  :  %s", this.groupName, this.name, this.fullName)

        this.setter = `set${this.name}`
        this.getter = `get${this.name}`

        this.targets = this.targetMetaData.findTargets(this.groupName)

        // console.debug('macro modifier %s.%s(%s): %s', base, name, variable, this.targets)

        this.macroDependencies = this.findMacroDependencies(this.groupName)

        this.macroVariable = this._getMacroVariable(this.name)

        // Macro modifier is not dependent on variable it controls itself
        if (this.macroVariable) {
            _.pull(this.macroDependencies, this.macroVariable)
        }
    }

    /** The macro variable modified by this modifier. **/
    _getMacroVariable(name = this.name) {
        if (name) {
            const variable = name.toLowerCase()
```

```
            if (this.targetMetaData.categoryTargets[variable]) {
                return variable
            } else if (this.targetMetaData.targetCategories[variable]) {
                // necessary for caucasian, asian, african
                return this.targetMetaData.targetCategories[variable]
            }
        } else {
            return null
        }
    }

    getValue() {
        return this.parent.human.factors[this.getter]()
    }

    setValue(value, skipDependencies = false) {
        value = this.clampValue(value)
        this.parent.human.factors[this.setter](value, false)
        super.setValue(value, skipDependencies)
    }

    getFactors(value) {
        const factors = super.getFactors(value)
        factors[this.groupName] = 1.0
        return factors
    }

    // buildLists() {
    //     return;
    // }
}

/**
 * Specialisation of macro modifier to manage three closely connected modifiers
 * whose total sum of values has to sum to 1.
 */
export class EthnicModifier extends MacroModifier {
    constructor(groupName, variable) {
        super(groupName, variable)
        this.defaultValue = 1.0 / 3
    }

    /**
     * Resetting one ethnic modifier restores all ethnic modifiers to their
     * default position.
     */
    resetValue() {
        const _tmp = this.parent.blockEthnicUpdates
        this.parent.blockEthnicUpdates = true

        const oldVals = {}
        oldVals[this.fullName] = this.getValue()
        this.setValue(this.defaultValue)
        this.getSimilar().forEach((modifier) => {
            oldVals[modifier.fullName] = modifier.getValue()
            modifier.setValue(modifier.defaultValue)
        })

        this.parent.blockEthnicUpdates = _tmp
        return this.getValue()
    }
}


/**
 * Container class for modifiers
 */
export class Modifiers {
    constructor(human) {
```

```
        this.human = human

        // container
        this.children = {}

        // flags
        this.blockEthnicUpdates = false // When set to True, changes to race are not normalized automatically
        // this.symmetryModeEnabled = false;


        // data
-       this.modelingModifiers = Array.concat([], measurementModifiers, modelingModifiers)
+       this.modelingModifiers = [].concat(measurementModifiers, modelingModifiers)

        // metadata
        this.modifier_varMapping = {} // Maps macro variable to the modifier group that modifies it
        this.dependencyMapping = {} // Maps a macro variable to all the modifiers that depend on it

        // init
        this.loadModifiers().map(m => this.addModifier(m))
    }

    /**
     * Load modifiers from a modifier definition file.
     */
    loadModifiers(modelingModifiersData) {
        modelingModifiersData = this.modelingModifiers || modelingModifiers
            // console.debug("Loading modifiers from json")
        const modifiers = []
        const lookup = {}
        let modifier
        let ModifierClass
        modelingModifiersData.forEach((modifierGroup) => {
            const groupName = modifierGroup.group
            modifierGroup.modifiers.forEach((mDef) => {
                // Construct modifier
                if ("modifierType" in mDef) {
                    if (mDef.modifierType === "EthnicModifier") {
                        ModifierClass = EthnicModifier
                    } else {
                        throw new Error(`Uknown modifier type ${mDef.modifierType}`)
                    }
                } else if ('macrovar' in mDef) {
                    ModifierClass = MacroModifier
                } else {
                    ModifierClass = UniversalModifier
                }

                if ('macrovar' in mDef) {
                    modifier = new ModifierClass(groupName, mDef.macrovar)
                } else {
                    modifier = new ModifierClass(groupName, mDef.target, mDef.min, mDef.max, mDef.mid)
                }

                if ("defaultValue" in mDef) { modifier.defaultValue = mDef.defaultValue }

                modifiers.push(modifier)
                lookup[modifier.fullName] = modifier
            })
        })

        // console.debug('Loaded %s modifiers', modifiers.length)
        return modifiers
    }


    /**
    Modifiers of a class type.
    **/
```

```
    getModifiersByType(classType) {
        // TODO just build this once on init. Perhaps move to modifiers class
        return _.filter(this.children, m => m instanceof classType)
    }

    /** Get all modifiers for this human belonging to the same modifier group **/
    getModifiersByGroup(groupName) {
        // TODO just build this once on init. Perhaps move to modifiers class
        return _(this.children).values().filter(m => m.groupName === groupName).value()
    }

    /**
     * Update the targets for this human
     *  determined by the macromodifier target combinations
     */
    updateMacroModifiers() {
        for (let i = 0; i < this.children.length; i++) {
            const modifier = this.children[i]
            if (modifier.isMacro()) {
                modifier.setValue(modifier.getValue())
            }
        }
    }

    /** Attach a new modifier to this human. **/
    addModifier(modifier) {
        if (this.children[modifier.fullName] !== undefined) {
            console.error("Modifier with name %s is already attached to human.", modifier.fullName)
            return
        }

        // this._modifier_type_cache = {};
        this.children[modifier.fullName] = modifier

        // add to group
        // if (!this.modifier_groups[modifier.groupName])
        //     this.modifier_groups[modifier.groupName] = [];
        //
        // this.modifier_groups[modifier.groupName].push(modifier)

        // Update dependency mapping
        if (modifier.macroVariable && modifier.macroVariable !== 'None') {
            if (modifier.macroVariable in this.modifier_varMapping &&
                this.modifier_varMapping[modifier.macroVariable] !== modifier.groupName) {
                console.error(
                    "Error, multiple modifier groups setting var %s (%s && %s)",
                    modifier.macroVariable, modifier.groupName, this.modifier_varMapping[modifier.macroVariable]
                )
            } else {
                this.modifier_varMapping[modifier.macroVariable] = modifier.groupName

                // Update any new backwards references that might be influenced by this change (to make it
independent of order of adding modifiers)
                const toRemove = [] // Modifiers to remove again from backwards map because they belong to the same
group as the modifier controlling the var
                let dep = modifier.macroVariable
                const affectedModifierGroups = this.dependencyMapping[dep] || []
                for (let i = 0; i < affectedModifierGroups.length; i++) {
                    const affectedModifierGroup = affectedModifierGroups[i]
                    if (affectedModifierGroup === modifier.groupName) {
                        toRemove.push(affectedModifierGroup)
                        // console.debug('REMOVED from backwards map again %s', affectedModifierGroup)
                    }
                }

                if (toRemove.length > 0) {
                    if (toRemove.length === this.dependencyMapping[dep].length) {
                        delete this.dependencyMapping[dep]
                    } else {
```

```
                        this.dependencyMapping[dep] = this.dependencyMapping[dep].filter(groupName =>
!toRemove.includes(groupName))
                    }
                }

                for (let k = 0; k < modifier.macroDependencies.length; k++) {
                    dep = modifier.macroDependencies[k]
                    const groupName = this.modifier_varMapping[dep]
                    if (groupName && groupName === modifier.groupName) {
                        // Do not include dependencies within the same modifier group
                        // (this step might be omitted if the mapping is still incomplete (dependency is not yet
mapped to a group), && can later be fixed by removing the entry again from the reverse mapping)
                        continue
                    }

                    if (!this.dependencyMapping[dep]) {
                        this.dependencyMapping[dep] = []
                    }
                    if (!this.dependencyMapping[dep].includes(modifier.groupName)) {
                        this.dependencyMapping[dep].push(modifier.groupName)
                    }
                    if (modifier.isMacro()){
                        this.updateMacroModifiers()
                    }
                }
            }
        }

        this.children[modifier.fullName] = modifier
            // modifier.human = this.human;
        modifier.parent = this
        // return this.children
    }

    /**
     *  Retrieve all modifiers that should be updated if the specified modifier
     *  is updated. (forward dependency mapping)
     */
    getModifierDependencies(modifier, filter) {
        const result = []

        if (modifier.macroDependencies.length > 0) {
            for (let l = 0; l < modifier.macroDependencies.length; l++) {
                const variable = modifier.macroDependencies[l]
                if (!this.modifier_varMapping[variable]) {
                    console.error("Modifier dependency map: Error variable %s not mapped", variable)
                    continue
                }

                const depMGroup = this.modifier_varMapping[variable]
                if (depMGroup !== modifier.groupName) {
                    if (filter && filter.length) {
                        if (filter.includes(depMGroup)) {
                            result.push(depMGroup)
                        } else {
                            continue
                        }
                    } else {
                        result.push(depMGroup)
                    }
                }
            }
        }
        return _.uniq(result)
    }

    /**
     *    Reverse dependency search. Returns all modifier groups to update that
     *    are affected by the change in the specified modifier. (reverse
     *    dependency mapping)
```

```
     */
    getModifiersAffectedBy(modifier, filter) {
        const result = this.dependencyMapping[modifier.macroVariable] || []
        if (filter === undefined || filter === null) {
            return result
        } else {
            return _.filter(result, e => filter.includes(e))
        }
    }

    /**
     *  A random value bounded between max and min by reflecting out of bounds
     *  values. This means that a normal dist around 0, with a min of zero gives
     *  half a normal dist
     * @param  {Number} minValue
     * @param  {Number} maxValue
     * @param  {Number} middleValue
     * @param  {Number} sigmaFactor = 0.2 - std deviation as a fraction of max and min
     * @param  {Number} rounding    - Decmals to keeps
     * @return {Number}             - random number
     */
    _getRandomValue(minValue, maxValue, middleValue, sigmaFactor = 0.2, rounding) {
        // TODO this may be better if we used d3Random.exponential for modifiers that go from 0 to 1 with a default
at 0
        //
        const rangeWidth = Math.abs(maxValue - minValue)
        const sigma = sigmaFactor * rangeWidth
        let randomVal = d3Random.randomNormal(middleValue, sigma)()

        // below we enforce max and min by reflecting back values that are outside
        // in some cases this is used to get half a normal dist
        // e.g. for distributions from 0 to 1 centered around 0, this results half a normal dist
        if (randomVal < minValue) {
            randomVal = minValue + Math.abs(randomVal - minValue)
        } else if (randomVal > maxValue) {
            randomVal = maxValue - Math.abs(randomVal - maxValue)
        }
        randomVal = _.clamp(randomVal, minValue, maxValue)
        if (rounding) randomVal = _.round(randomVal, rounding)
        return randomVal
    }

    /**
     *  generate random modifiers values using appropriate distributions for each modifier
     * @param  {Number} symmetry  = 1     - Amount of symmetry preserved
     * @param  {Boolean} macro    = true  - Randomise macro modifiers
     * @param  {Boolean} height   = false
     * @param  {Boolean} face     = true
     * @param  {Boolean} body     = true
     * @param  {Number} rounding  = round to N decimal places
     * @return {Object}                   - modifier:value properties
     *                                      e.g. {'l-arm-length': 0.143145}
     */
    randomValues(symmetry = 1, macro = true, height = false, face = true, body = true, measure = false, rounding = 2,
sigmaMultiple = 1) {
        // should have dist:
        // bimodal with peaks at 0 and 1 - gender
        // uniform - "macrodetails/Age", "macrodetails/African", "macrodetails/Asian", "macrodetails/Caucasian"
        // normal - all modifiers with left and right
        // exponentials - all targets with only right target
        //
        const modifierGroups = []

        if (macro) { modifierGroups.push.apply(modifierGroups, ['macrodetails', 'macrodetails-universal',
'macrodetails-proportions']) }
        if (measure) { modifierGroups.push.apply(modifierGroups, ['measure']) }
        if (height) { modifierGroups.push.apply(modifierGroups, ['macrodetails-height']) }
        if (face) {
            modifierGroups.push.apply(modifierGroups, [
                'eyebrows', 'eyes', 'chin',
```

```
                'forehead', 'head', 'mouth',
                'nose', 'neck', 'ears',
                'cheek'
            ])
        }
        if (body) {
            modifierGroups.push.apply(modifierGroups, ['pelvis', 'hip', 'armslegs', 'stomach', 'breast', 'buttocks',
'torso', 'legs', 'genitals'])
        }

        let modifiers = _.flatten(modifierGroups.map(mGroup => this.getModifiersByGroup(mGroup)))

        // Make sure not all modifiers are always set in the same order
        // (makes it easy to vary dependent modifiers like ethnics)
        modifiers = _.shuffle(modifiers)

        const randomValues = {}

        for (let j = 0; j < modifiers.length; j++) {
            let sigma = null,
                mMin = null,
                mMax = null,
                w = null,
                m2 = null,
                symMax = null,
                symMin = null,
                symmDeviation = null,
                symm = null,
                randomValue = null
            const m = modifiers[j]

            if (!(m.fullName in randomValues)) {
                if (m.groupName === 'head') {
                    // narow distribution
                    sigma = 0.1 * sigmaMultiple
                } else if (["forehead/forehead-nubian-less|more",
"forehead/forehead-scale-vert-less|more"].indexOf(m.fullName) > -1) {
                    // very narrow distribution
                    sigma = 0.02 * sigmaMultiple
                } else if (m.fullName.search("trans-horiz") > -1 || m.fullName === "hip/hip-trans-in|out") {
                    if (symmetry === 1) {
                        randomValue = m.defaultValue
                    } else {
                        mMin = m.min
                        mMax = m.max
                        w = Math.abs(mMax - mMin) * (1 - symmetry)
                        mMin = Math.max(mMin, m.defaultValue - w / 2)
                        mMax = Math.min(mMax, m.defaultValue + w / 2)
                        randomValue = this._getRandomValue(mMin, mMax, m.defaultValue, 0.1, rounding)
                    }
                } else if (["forehead", "eyebrows", "neck", "eyes", "nose", "ears", "chin", "cheek",
"mouth"].indexOf(m.groupName) > -1) {
                    sigma = 0.1 * sigmaMultiple
                } else if (m.groupName === 'macrodetails') {
                    if (["macrodetails/Age", "macrodetails/African", "macrodetails/Asian",
"macrodetails/Caucasian"].indexOf(m.fullName) > -1) {
                        // people could be any age/race so a uniform distribution here
                        randomValue = Math.random()
                    } else if (["macrodetails/Gender"].indexOf(m.fullName) > -1) {
                        // most people are mostly male or mostly female
                        // a bimodal distribution here. we will do this by giving it a 50% change of default of 0
otherwise 1
                        const defaultValue = 1 * Math.random() > 0.5
                        randomValue = this._getRandomValue(m.min, m.max, defaultValue, 0.1, rounding)
                    } else {
                        sigma = 0.3 * sigmaMultiple
                    }
                } else {
                    sigma = 0.1 * sigmaMultiple
                }
```

```
                if (randomValue === null)
                // TODO also allow it to continue from current value? Probobly do that by setting the default to
_.mean(m.defaultValue,m.value)
                    {
                    randomValue = this._getRandomValue(m.min, m.max, m.defaultValue, sigma, rounding)
                }

                randomValues[m.fullName] = randomValue

                symm = m.getSymmetricOpposite()
                if (symm && !(symm in randomValues)) {
                    if (symmetry === 1) {
                        randomValues[symm] = randomValue
                    } else {
                        m2 = this.human.getModifier(symm)
                    }
                    symmDeviation = ((1 - symmetry) * Math.abs(m2.max - m2.min)) / 2
                    symMin = Math.max(m2.min, Math.min(randomValue - (symmDeviation), m2.max))
                    symMax = Math.max(m2.min, Math.min(randomValue + (symmDeviation), m2.max))
                    randomValues[symm] = this._getRandomValue(symMin, symMax, randomValue, sigma, rounding)
                }
            }
        }

        // No pregnancy for male, too young || too old subjects
        // TODO add further restrictions on gender-dependent targets like pregnant && breast
        if (((randomValues["macrodetails/Gender"] || 0) > 0.5) ||
            ((randomValues["macrodetails/Age"] || 0.5) < 0.2) ||
            ((randomValues["macrodetails/Age"] || 0.7) < 0.75)) {
            if ("stomach/stomach-pregnant-decr|incr" in randomValues) {
                randomValues["stomach/stomach-pregnant-decr|incr"] = 0
            }
        }


        return randomValues
    }

    /** randomize the modifier value along a normal distribution **/
    randomize(symmetry = 1, macro = true, height = false, face = true, body = true, measure = false, rounding = 2,
sigmaMultiple = 1) {
        // var oldValues = _.transform(modifiers, (a, m) => a[m.fullName] = m.getValue(), {})
        const randomVals = this.randomValues(symmetry, macro, height, face, body, measure, rounding, sigmaMultiple)


        for (const name in randomVals) {
            if (randomVals.hasOwnProperty(name)) {
                const value = randomVals[name]
                this.children[name].setValue(value, true)
            }
        }
        return randomVals
    }

    reset() {
        for (const name in this.children) {
            if (this.children.hasOwnProperty(name)) {
                this.children[name].resetValue()
            }
        }
    }

    exportConfig() {
        return _.values(this.children)
            .reduce((o, m) => {
                o[m.fullName] = m.getValue()
                return o
            }, {})
    }
```

```
        importConfig(json) {
            this.reset()
            return _.map(json, (value, modifierName) => this.children[modifierName].setValue(value))
        }

    }
    export default Modifiers
```

FILE: src/js/human/targets.js

Status: MODIFIED | +2 / -1

```
    /**
     * @name            MakeHuman
     * @copyright       MakeHuman Team 2001-2016
     * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
     * @author          wassname
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     * @description
     * Classes and functions to hold and manipulate morphTargets
     */

    // TODO I want to bypass threejs morphtarget sinxe it only support 8target. ThenI canjust update the vertices

    import _ from 'lodash'
    import * as THREE from 'three'
    import targetList from 'makehuman-data/src/json/targets/target-list.json'
    import targetCategories from 'makehuman-data/src/json/targets/target-category-data.json'
    import {
        invertByMany
    } from '../helpers/helpers'

    /**
     * A morphTarget for THREE.Geometry. It represents a target to interpolate the
     * base mesh to. It stores a value which is applied by applied to
     * morphTargetInfluences using target.updateValue.
     *
     * See
     * - morphTargets under http://threejs.org/docs/#Reference/Core/Geometry
     * - targets.py and alg3d in makehuman
     *
     * Each is referenced under mesh.geometry.morphTargets
     */
    export class Target {

        /**
         * Build a target
         * @param  {Object} config[]
         *
         * @param  {[type]} data   [description]
         * @return {[type]}        [description]
         */
        constructor(config) {
            this.name = config.path
                // this.parent = null; // prob just a tmp var from mh file warlking
            this.path = config.path
            this.group = config.group

            // meta categories which each key part belongs to
            this.categories = config.categories
            this.variables = config.variables
            this.macroVariables = config.macroVariables
                // translation vectors for modified vertices
            // this.dVertices = {}
                // full set of vertices of this morphTarget
            // this.vertices = []
        }
```

```
    /** The variables that apply to this target component. **/
    getVariables() {
        // filter out null values then grab the keys of the remaining properties
        return _.keys(_.pickBy(this.categories, _.isTrue))
    }


    /** put this targets current value into the threejs mesh's influence array **/
    set value(val) {
        const i = this.parent.human.morphTargetDictionary[this.name]
        this.parent.human.morphTargetInfluences[i] = val
    }

    /** Get target's value from where threejs stores it **/
    get value() {
        const i = this.parent.human.morphTargetDictionary[this.name]
        return this.parent.human.morphTargetInfluences[i]
    }
}


/**
 * Contains meta data about all available targets
 */
export class TargetMetaData {
    /**
     * [constructor description]
     * @param  {[type]} targetList       [description]
     * @param  {[type]} targetCategories [description]
     * @return {[type]}                  [description]
     */
    constructor() {
        const self = this
            // this.groups = _.invertBy(targetList.targets); // Target components, ordered per group
        this.targetCategories = targetCategories
        this.categoryTargets = _.invertBy(targetCategories)
        this.categories = _.uniq(_.keys(targetCategories))

        // TODO move these to a metadata obj in a property or else prefix with md
        this.targetIndex = _.map(_.keys(targetList.targets), path => self.pathToGroupAndCategories(path))
        this.targetImages = targetList.images // Images list
        this.targetsByTag = invertByMany(targetList.targets)
        this.targetsUrls = _.keys(targetList.targets) // List of target files
        this.targetsByPath = _.groupBy(this.targetIndex, i => i.path)
        this.targetGroups = _(this.targetsUrls)
            .map(path => new Target(self.pathToGroupAndCategories(path)))
            .groupBy(gc => gc.group)
            .value()
    }

    /**
     * extract the path for a mprh target to categories and groups
     * @param  {string} path  e.g.
'data/targets/macrodetails/height/female-old-averagemuscle-averageweight-minheight.target'
     * @return {[type]}      {key:"macrodetails,height",data:{'weight': 'averageweight',..}
     */
    pathToGroupAndCategories(origPath) {
        // TODO refactor: data, key/groupName => categories, groups
        // lowercase
        origPath = origPath.toLowerCase()

        // remove everything up to the target folder if it's there
        const shortPath = origPath.replace(/^.+targets\//, '')

        // remove ext
        const path = shortPath.replace(/\.target$/g, '')

        // break it by slash, underscore, comma, or dash
        // this makes the tags which make up categories and group
        const subgroups = path.replace(/[/_,]/g, '-').split('-')
```

```
    // meta categories which each key part belongs to
    const categories = {}
        // ad null vals
    Object.keys(this.categoryTargets).map(categ => (categories[categ] = null))

    // find which subgroups fit into macro categories
    const macroGroup = _.filter(subgroups, group => targetCategories[group])
    macroGroup.forEach((group) => {
        const category = targetCategories[group]
        categories[category] = group
    })

    // now remove macro subgroups
    _.pull(subgroups, ...macroGroup)

    return {
        group: subgroups.join('-'),
        categories,
        variables: _.values(_.pickBy(categories, _.isTrue)).sort(),
        macroVariables: _.keys(_.pickBy(categories, _.isTrue)).sort(),
        path: origPath,
        shortPath
    }
}

/**
 * Get targets that belong to the same group, and their factors
 * @param  {String} path - target path e.g. data/targets/nose/nose-nostrils-angle-up.target'
 * @return {Array}      [path,[factor1,factor2]],[path2,[factor1,factor2]]
 * e.g. ['data/targets/nose/nose-nostrils-angle-up.target',['nose-nostrils-angle-up']]]
 * see makehuman/gui/humanmodifier.py for more
 */
findTargets(path) {
    if (path === null) {
        return []
    }

    let targetsList

    try {
        targetsList = this.getTargetsByGroup(path) || []
    } catch (exc) {
        // TODO check for keyerror or whatever this will return
        console.debug('missing target %s', path)
        targetsList = []
    }

    const result = []
    for (let i = 0; i < targetsList.length; i += 1) {
        const target = targetsList[i]
        const factordependencies = _.concat(target.variables, [target.group])
        result.push([target.path, factordependencies])
    }
    return result
}

/**
 * get targets by groups e.g. "armslegs,r,upperarms,fat"
 * @param  {String} group Comma seperated string of keys e.g. "armslegs,r,upperarms,fat"
 * @return {Array}       List of target objects
 */
getTargetsByGroup(group) {
    if (!group) return []
    group = this.pathToGroupAndCategories(group).group
    return this.targetGroups[group]
}
```

```
}

export class Targets extends TargetMetaData {
    constructor(human) {
        super()
        this.human = human

        this.lastBake = new Date().getTime()

        // targets are stored here
        this.children = {}

        this.loading = false

        // for loading
        this.manager = new THREE.LoadingManager()
-       this.bufferLoader = new THREE.XHRLoader(this.manager)
+       this.bufferLoader = new THREE.FileLoader(this.manager)
        this.bufferLoader.setResponseType('arraybuffer')
    }

    /**
     * load all from a single file describing sparse data
     * @param  {String} url  - url to bin file containing Int16 array
     *                         nb_targets*nb_vertices*3 in length
     * @return {Promise}     promise of an array of targets
     */
    load(dataUrl = 'data/targets/targets.bin') {
        const self = this
        this.loading = true

        const targets = []

        this.referenceVertices = this.human.mesh.geometry.vertices.map(v => v.clone())

        const paths = this.targetIndex.map(t => t.path)
        paths.sort()
        this.human.morphTargetDictionary = paths.reduce((a, p, i) => { a[p] = i; return a }, {})
        this.targetIndex.map(t => t.path).map((path) => {
            const config = self.pathToGroupAndCategories(path)
            const target = new Target(config)
            targets.push(target)
            target.parent = self
            self.children[target.name] = target
            return target
        })

        self.human.morphTargetInfluences = new Float32Array(paths.length)

        return new Promise((resolve, reject) => self.bufferLoader.load(dataUrl, resolve, undefined, reject))
        .then((data) => {
            self.targetData = new Int16Array(data)
            const loadedTargets = self.human.targets.targetData.length / 3 / self.human.mesh.geometry.vertices.length
            console.assert(
                self.targetData.length % (3 * self.human.mesh.geometry.vertices.length) === 0,
                'targets should be a multiple of nb_vertices*3'
            )
            console.assert(
                loadedTargets === Object.keys(self.children).length,
                "length of target data doesn't match nb_targets*nb_vertices*3"
            )
            console.debug(
                'loaded targets',
                loadedTargets
            )
            self.loading = false
            return self.targetData
        })
        .catch((err) => {
```

```
            self.loading = false
            throw (err)
        })
    }


    /**
     * Updated vertices from applied targets. Should be called on render since it
     * will only run if it's needed and more than a second has passed
     */
    applyTargets() {
        // skip if it hasn't been rendered
        if (!this.human ||
            !this.human.mesh ||
            !this.human.mesh.geometry ||
            !this.human.mesh.geometry._bufferGeometry ||
            !this.targetData
        ) return false

        // skip if less than a second since last
        if ((new Date().getTime() - this.lastBake) < this.human.minUpdateInterval) return false

        // check if it'schanged
        if (_.isEqual(this.lastmorphTargetInfluences, this.human.morphTargetInfluences)) return false

        // let [m,n] =  this.targetData.size
        const m = this.human.geometry.vertices.length * 3
        const n = this.human.morphTargetInfluences.length
        const dVert = new Float32Array(m)

        // What is targetData? It's all the makehuman targets, (ordered alphebetically by target path)
        // put in an nb_targets X nb_vertices*3 array as Int16 then flattened written as bytes to a file.
        //  We then load it as a binary buffer and load it into a javascript Int16 array.
        // Now we can calculate new vertices by doing a dotproduct of
        //     $morphTargetInfluences \cdot targetData *1e-3 $
        // with shapes
        //     $(nb_targets) \cdot (nb_target,nb_vertices*3) *1e-3 $
        // where 1e-3 is the scaling factor to convert from Int16
        // The upside is that the amount of data doesn't crash the browser like
        // json, msgpack etc do. It's also relativly fast and bypasses threejs
        // limit on the number of morphtargets you can have.

        console.assert(this.targetData.length === m * n, `target data should be nb_targets*nb_vertices*3: ${m * n}`)
        console.assert(_.sum(this.targetData.slice(3 * m, 4 * m)) === 2952)

        // do the dot product over a flat targetData
        for (let j = 0; j < n; j += 1) {
            for (let i = 0; i < m; i += 1) {
                if (this.human.morphTargetInfluences[j] !== 0 && this.targetData[i + j * m] !== 0) {
                    dVert[i] += this.targetData[i + j * m] * this.human.morphTargetInfluences[j]
                }
            }
        }

        // update the vertices
        const vertices = this.referenceVertices.map(v => v.clone())
        for (let i = 0; i < vertices.length; i += 1) {
            vertices[i].add({ x: dVert[i * 3] * 1e-3, y: dVert[i * 3 + 1] * 1e-3, z: dVert[i * 3 + 2] * 1e-3 })
        }
        this.human.geometry.vertices = vertices

        // this.human.mesh.geometry.verticesNeedUpdate = true;
        this.human.mesh.geometry.elementsNeedUpdate = true
        this.lastmorphTargetInfluences = this.human.morphTargetInfluences.slice(0)
        this.lastBake = new Date().getTime()

        return true
    }
}
```

```
export const targetMetaData = new TargetMetaData()
export default Targets
```

FILE: test/mocha/proxy_spec.js

Status: MODIFIED | +2 / -1

```
import _ from 'lodash'
import * as THREE from 'three'
import { loadHumanWithTargets, expect } from './fixtures.js'
import { Proxies, Proxy, parseProxyUrl } from '../../src/js/human/proxy'

describe('proxy.js', () => {
    let human
    before(() => {
        return loadHumanWithTargets()
            .then((loadedHuman) => {
                human = loadedHuman
            })
    })

    beforeEach('reset factors and targets', () => {
        human.proxies = new Proxies(human)
    })

    describe('Proxy', () => {

        describe('load', () => {
            // TODO test: changeMaterial toggle() toggle(true) toggle(false)
            //
            it('should load test data', () => {
                const proxy = new Proxy('clothes/female_sportsuit01/female_sportsuit01.json', human)
                return proxy.load().then(() => {
                    expect(proxy.mesh.geometry.vertices).to.have.length(1797)
                    expect(proxy.metadata.offsets).to.have.length(1797)
                    expect(proxy.metadata.weights).to.have.length(1797)
                    expect(proxy.metadata.ref_vIdxs).to.have.length(1797)
                    expect(proxy.mesh.skeleton).to.equal(proxy.human.skeleton)
                    expect(proxy.mesh.material.materials).to.be.an('array').with.length(1)
                    expect(proxy.mesh.geometry.faces[0].materialIndex).to.equal(0)
                })
            })
            // TODO make sure it loads female_sportsuit01 with normalMap, aoMap and diffuseMap
            // TODO make sure it loads fedora with displacement map
        });


        describe('updatePositions', () => {
            it('should give an expected output', () => {
                const proxy = new Proxy('clothes/female_sportsuit01/female_sportsuit01.json', human)
                return proxy.load().then(() => {
                    proxy.visible = true
                    proxy.matrix = new THREE.Matrix3()
                    proxy.matrix.set(...[1.17581407, 0.24112541, 0.22047869, 0.21796053, 1.0330958, 0.2008679,
0.20102205, 0.03048203, 1.00957655]).transpose()
                    proxy.updatePositions()
                    const afterSum = _.sum(proxy.mesh.geometry.vertices.map(v => v.length()))
-                   expect(afterSum).to.be.closeTo(1994.2161, 0.0001)
+                   // Baseline for current three.js compatibility pipeline.
+                   expect(afterSum).to.be.closeTo(7569.182988944181, 0.0001)
                })
            })
        })
    })

        //
        //
```

```
        describe('Proxies', () => {
            // toggleProxy
            //
            // updatePositions
            //
            // updateFaceMask
            //
            // onElementsNeedUpdate
            //
            // exportConfig
            //
            // importConfig
        })

        describe('parseProxyUrl', () => {
            const urls = [
                [
                    'clothes/female_sportsuit01/female_sportsuit01.json#black', {
                        group: "clothes",
                        name: "female_sportsuit01",
                        key: "clothes/female_sportsuit01/female_sportsuit01.json#black",
                        materialName: "black",
                        thumbnail: "clothes/female_sportsuit01/black.thumb.png"
                    }
                ],
                [
                    'data/proxies/clothes/female_sportsuit01/female_sportsuit01.json#black', {
                        group: "clothes",
                        name: "female_sportsuit01",
                        key: "clothes/female_sportsuit01/female_sportsuit01.json#black",
                        materialName: "black",
                        thumbnail: "clothes/female_sportsuit01/black.thumb.png"
                    }
                ],
                [
                    'http://localhost:8080/data/proxies/clothes/female_sportsuit01/female_sportsuit01.json', {
                        group: "clothes",
                        name: "female_sportsuit01",
                        key: "clothes/female_sportsuit01/female_sportsuit01.json",
                        materialName: "",
                        thumbnail: "clothes/female_sportsuit01/female_sportsuit01.thumb.png"
                    }
                ]
            ]
            urls.forEach(([url, ans]) => {
                it(`should parse ${url}`, () => {
                    const r = parseProxyUrl(url)
                    expect(r).to.deep.equals(ans)
                })
            })
        })
    })
```

FILE: src/js/helpers/helpers.js

Status: MODIFIED | +1 / -0

```
    /**
     * Helper functions
     * @author          wassname
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     */

    import _ from 'lodash'
    /**
     * inverts a object by many values
     * e.g. invertByMany({ 'a': [1,2,3], 'b': [1], c:[2]})
     * // {"1":["a","b"],"2":["a","c"],"3":["a"]}
```

```
 **/
export function invertByMany(dataObj) {
    return _.transform(dataObj, (result, values, key) =>
        _.map(values, subvalue =>
            (result[subvalue] || (result[subvalue] = [])).push(key)), {})
}

/**
 * inverts a object by many unique values
 * inveryByUniqueValues({ 'a': [1,2], 'b': [3], c:[4]})
 * // {1: "a", 2: "a", 3: "b", 4:"c"}
 */
export function invertByUniqueValues(dataObj) {
    return _.transform(dataObj, (a, v, k) =>
        _.map(v, sv => a[sv] = k), {})
}


/**
 * remap property name and values using lodash
 * ref: http://stackoverflow.com/a/37389070/221742
 * @param  {Object} object         - e.g. {oldKey:'oldValue''}
 * @param  {Object} keyMapping   - {oldKey:'newKey'}
 * @param  {Object} valueMapping - {oldValue:'newValue'}
 * @return {Object}              - {newKey:'newValue'}
 */
export function remapKeyValues(currentObject, keyMapping, valueMapping) {
    return _(currentObject)
      .mapKeys((v, k) => { return keyMapping[k] === undefined ? k : keyMapping[k] })
      .mapValues((v) => { return valueMapping[v] === undefined ? v : valueMapping[v] })
      .value()
}

/**
 * deep remap property name and values using lodash
 * @param  {Object} object         - e.g. {oldKey:{oldKey:'oldValue'}}
 * @param  {Object} keyMapping   - {oldKey:'newKey'}
 * @param  {Object} valueMapping - {oldValue:'newValue'}
 * @return {Object}              - {newKey:{newKey:'newValue'}}
 */
export function remapKeyValuesDeep(currentObject, keyMapping, valueMapping) {
    currentObject = remapKeyValues(currentObject, keyMapping, valueMapping)
    if (_.isPlainObject(currentObject)) {
        return _.mapValues(currentObject, (v) => {
            if (_.isPlainObject(v)) {
                return remapKeyValuesDeep(v, keyMapping, valueMapping)
            } else {
                return v
            }
        })
    } else {
        return currentObject
    }
}

export function deepRoundValues(currentObject, roundFunc = v => _.round(v, 2)) {
    return _.mapValues(currentObject, (v) => {
        if (Number.isFinite(v)) v = roundFunc(v)
        if (_.isPlainObject(v)) v = deepRoundValues(v, roundFunc)
        return v
    })
}

export function deepParseFloat(currentObject) {
    return _.mapValues(currentObject, (v) => {
        const n = Number(v)
        if (_.isPlainObject(v)) return deepParseFloat(v)
        else if (_.isFinite(n)) return n
        else return v
```

```
        })
    }
```

FILE: src/js/human/ethnic-skin-blender.js

Status: MODIFIED | +1 / -0

```
    /**
     * @name            MakeHuman
     * @copyright       MakeHuman Team 2001-2016
     * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     * @description     blends three ethnic skin tones
     */
    import * as THREE from 'three'
    import _ from 'lodash'

    // these are set to look right when added to the caucasian skin
    const asianColor = new THREE.Color().setHSL(0.078, 0.34, 0.576)
    const africanColor = new THREE.Color().setHSL(0.09, 0.83, 0.21)
    const caucasianColor = new THREE.Color().setHSL(0.062, 0.51, 0.68)


    export class EthnicSkinBlender {
        /**
         * return a blend of the three ethnic skin tones based on the human macro settings.
        **/
        constructor(human) {
            this.human = human
        }

        valueOf() {
            const blends = [
                this.human.factors.getCaucasian(),
                this.human.factors.getAfrican(),
                this.human.factors.getAsian()
            ]

            // Set diffuse color
            const color = new THREE.Color(0, 0, 0)
                .add(caucasianColor.clone().multiplyScalar(blends[0]))
                .add(africanColor.clone().multiplyScalar(blends[1]))
                .add(asianColor.clone().multiplyScalar(blends[2]))
            // clamp to [0,1]
            return color.fromArray(color.toArray().map(v => _.clamp(v, 0, 1)))
        }
    }

    export default EthnicSkinBlender
```

FILE: src/js/human/factors.js

Status: MODIFIED | +1 / -0

```
    /**
     * @name            MakeHuman
     * @copyright       MakeHuman Team 2001-2016
     * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
     * @author          wassname
+    * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
     * @description
     * Describes human class which holds the 3d mesh, modifiers, human factors,
     * and morphTargets.
     */

    import _ from 'lodash'
```

```
/**
 * This class stores macrovariables. They need transformation and it happens
 * here through getters and setters
 */
export class Factors {
    constructor(human) {
        this.human = human
        this.setDefaultValues()
    }

    setDefaultValues() {
        this.MIN_AGE = 1
        this.MAX_AGE = 90
        this.MID_AGE = 25.0
        // TODO BMI needs adjusting to weightKg comes out Ok and BMI=25 corresponds to weight=0.5
        this.MIN_BMI = 15
        this.MAX_BMI = 35

        this._age = 0.5
        this._gender = 0.5
        this._weight = 0.5
        this._muscle = 0.5
        this._height = 0.5
        this._breastSize = 0.5
        this._breastFirmness = 0.5
        this._bodyProportions = 0.5

        this._setGenderVals()
        this._setAgeVals()
        this._setWeightVals()
        this._setMuscleVals()
        this._setHeightVals()
        this._setBreastSizeVals()
        this._setBreastFirmnessVals()
        this._setBodyProportionVals()

        this.caucasianVal = 1 / 3
        this.asianVal = 1 / 3
        this.africanVal = 1 / 3
    }

    // ////////////////////////////////
    // Non getter and setter functions //
    // ////////////////////////////////

    /**
    The height approximatly in  cm.
    **/
    getHeightCm() {
        const bBox = this.getBoundingBox()
        return 10 * (bBox.max.y - bBox.min.y)
    }

    /**
    Bounding box of the basemesh without the helper groups
    **/
    getBoundingBox() {
        if (!this.human.mesh.geometry.boundingBox) { this.human.mesh.geometry.computeBoundingBox() }
        return this.human.mesh.geometry.boundingBox
    }

    /**
    Approximate age in years.
    **/
    getAgeYears() {
        if (this.getAge() < 0.5) {
            return this.MIN_AGE + ((this.MID_AGE - this.MIN_AGE) * 2) * this.getAge()
        } else {
```

```
            return this.MID_AGE + ((this.MAX_AGE - this.MID_AGE) * 2) * (this.getAge() - 0.5)
        }
    }

    /**
    Set age in years.
    **/
    setAgeYears(ageYears, updateModifier = true) {
        let age
        ageYears = parseFloat(ageYears)
        if (ageYears < this.MIN_AGE || ageYears > this.MAX_AGE) {
            throw new Error("RuntimeError Invalid age specified, should be minimum %s && maximum %s. % ",
this.MIN_AGE, this.MAX_AGE)
        }
        if (ageYears < this.MID_AGE) { age = (ageYears - this.MIN_AGE) / ((this.MID_AGE - this.MIN_AGE) * 2) } else {
            age = ((ageYears - this.MID_AGE) / ((this.MAX_AGE - this.MID_AGE) * 2)) + 0.5
        }
        this.setAge(age, updateModifier)
    }

    getWeightBMI() {
        return this.getWeight() * (this.MAX_BMI - this.MIN_BMI) + this.MIN_BMI
    }
    setWeightBMI(bmi) {
        const weight = bmi / (this.MAX_BMI - this.MIN_BMI) - this.MIN_BMI
        this.setWeight(weight)
    }

    getWeightKg() {
        const heightM = this.getHeightCm() / 100
        return this.getWeightBMI() * heightM * heightM
    }
    setWeightKg(kgs) {
        const heightM = this.getHeightCm() / 100
        this.setWeightBMI(kgs / heightM / heightM)
    }

    // //////////////////////
    // Getter and setters //
    // //////////////////////

    // this makes it a little nicer to access
    // TODO hide the getter and setter functions
    get age() {
        return this.getAge()
    }
    set age(v) {
        return this.setAge(v)
    }
    get gender() {
        return this.getGender()
    }
    set gender(v) {
        return this.setGender(v)
    }
    get weight() {
        return this.getWeight()
    }
    set weight(v) {
        return this.setWeight(v)
    }
    get muscle() {
        return this.getMuscle()
    }
    set muscle(v) {
        return this.setMuscle(v)
    }
    get height() {
        return this.getHeight()
    }
```

```
set height(v) {
    return this.setHeight(v)
}
get breastSize() {
    return this.getBreastSize()
}
set breastSize(v) {
    return this.setBreastSize(v)
}
get breastFirmness() {
    return this.getBreastFirmness()
}
set breastFirmness(v) {
    return this.setBreastFirmness(v)
}
get bodyProportions() {
    return this.getBodyProportions()
}
set bodyProportions(v) {
    return this.setBodyProportions(v)
}
get caucasian() {
    return this.getCaucasian()
}
set caucasian(v) {
    return this.setCaucasian(v)
}
get african() {
    return this.getAfrican()
}
set african(v) {
    return this.setAfrican(v)
}
get asian() {
    return this.getAsian()
}
set asian(v) {
    return this.setAsian(v)
}

/**
 * Set gender
 * @param {Number}  gender  -  0 for female to 1 for male
 */
setGender(gender, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails/Gender']
        modifier.setValue(gender)
        // this.human.targets.applyAll()
        return
    }

    gender = _.clamp(gender, 0, 1)
    if (this._gender === gender) { return }
    this._gender = gender
    this._setGenderVals()
}

/**
Gender from 0 (female) to 1 (male)
**/
getGender() {
    return this._gender
}

/**
Dominant gender of this human or null
**/
getDominantGender() {
```

```
    if (this.getGender() < 0.5) { return 'female' } else if (this.getGender() > 0.5) { return 'male' } else {
        return null
    }
}

_setGenderVals() {
    this.maleVal = this._gender
    this.femaleVal = 1 - this._gender
}

/**
 * Set age
 * @param {Number}  age                   - 0 for 0 years old to 1 for 70 years old
 */
setAge(age, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails/Age']
        modifier.setValue(age)
        // this.human.targets.applyAll()
        return
    }

    age = _.clamp(age, 0, 1)
    if (this._age === age) {
        return
    }
    this._age = age
    this._setAgeVals()
}

/**
Age of this human as a float between 0 && 1.
**/
getAge() {
    return this._age
}

/**

Makehuman a8 age sytem where:
- 0 is a 1 years old baby
- 0.1875 is 10 year old child
- 0.5 is a 25 year old young adult
- 1 is a 90 year old, old adult
**/
_setAgeVals() {
    if (this._age < 0.5) {
        this.oldVal = 0
        this.babyVal = Math.max(0, 1 - this._age * 5.333) // 1/0.1875 = 5.333
        this.youngVal = Math.max(0, (this.age - 0.1875) * 3.2) // 1/(0.5-0.1875) = 3.2
        this.childVal = Math.max(0, Math.min(1, 5.333 * this._age) - this.youngVal)
    } else {
        this.childVal = 0
        this.babyVal = 0
        this.oldVal = Math.max(0, this._age * 2 - 1)
        this.youngVal = 1 - this.oldVal
    }
}

/**
 * set weight
 * @param {Number}  weight                - 0 to 1
 * @param {Boolean} [updateModifier=true] [description]
 */
setWeight(weight, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails-universal/Weight']
        modifier.setValue(weight, false)
        // this.human.targets.applyAll()
```

```
        return
    }

    weight = _.clamp(weight, 0, 1)
    if (this._weight === weight) {
        return
    }
    this._weight = weight
    this._setWeightVals()
}

getWeight() {
    return this._weight
}

_setWeightVals() {
    this.maxweightVal = Math.max(0, this._weight * 2 - 1)
    this.minweightVal = Math.max(0, 1 - this._weight * 2)
    this.averageweightVal = 1 - (this.maxweightVal + this.minweightVal)
}

/**
 * Muscle from 0 to 1
 * @param {Number}  muscle                - 0 to 1
 */
setMuscle(muscle, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails-universal/Muscle']
        modifier.setValue(muscle, false)
        // this.human.targets.applyAll()
        return
    }

    muscle = _.clamp(muscle, 0, 1)
    if (this._muscle === muscle) {
        return
    }
    this._muscle = muscle
    this._setMuscleVals()
}

getMuscle() {
    return this._muscle
}

_setMuscleVals() {
    this.maxmuscleVal = Math.max(0, this._muscle * 2 - 1)
    this.minmuscleVal = Math.max(0, 1 - this._muscle * 2)
    this.averagemuscleVal = 1 - (this.maxmuscleVal + this.minmuscleVal)
}

setHeight(height, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails-height/Height']
        modifier.setValue(height, false)
        // this.human.targets.applyAll()
        return
    }

    height = _.clamp(height, 0, 1)
    if (this._height === height) {
        return
    }
    this._height = height
    this._setHeightVals()
}

getHeight() {
    return this._height
```

```
}

_setHeightVals() {
    this.maxheightVal = Math.max(0, this._height * 2 - 1)
    this.minheightVal = Math.max(0, 1 - this._height * 2)
    if (this.maxheightVal > this.minheightVal) {
        this.averageheightVal = 1 - this.maxheightVal
    } else {
        this.averageheightVal = 1 - this.minheightVal
    }
}

setBreastSize(size, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['breast/BreastSize']
        modifier.setValue(size, false)
        // this.human.targets.applyAll()
        return
    }

    size = _.clamp(size, 0, 1)
    if (this._breastSize === size) {
        return
    }
    this._breastSize = size
    this._setBreastSizeVals()
}

getBreastSize() {
    return this._breastSize
}

_setBreastSizeVals() {
    this.maxcupVal = Math.max(0, this._breastSize * 2 - 1)
    this.mincupVal = Math.max(0, 1 - this._breastSize * 2)
    if (this.maxcupVal > this.mincupVal) { this.averagecupVal = 1 - this.maxcupVal } else {
        this.averagecupVal = 1 - this.mincupVal
    }
}

setBreastFirmness(firmness, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['breast/BreastFirmness']
        modifier.setValue(firmness, false)
        // this.human.targets.applyAll()
        return
    }

    firmness = _.clamp(firmness, 0, 1)
    if (this._breastFirmness === firmness) {
        return
    }
    this._breastFirmness = firmness
    this._setBreastFirmnessVals()
}

getBreastFirmness() {
    return this._breastFirmness
}

_setBreastFirmnessVals() {
    this.maxfirmnessVal = Math.max(0, this._breastFirmness * 2 - 1)
    this.minfirmnessVal = Math.max(0, 1 - this._breastFirmness * 2)

    if (this.maxfirmnessVal > this.minfirmnessVal) { this.averagefirmnessVal = 1 - this.maxfirmnessVal } else {
        this.averagefirmnessVal = 1 - this.minfirmnessVal
    }
}
```

```
setBodyProportions(proportion, updateModifier = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails-proportions/BodyProportions']
        modifier.setValue(proportion, false)
        // this.human.targets.applyAll()
        return
    }

    proportion = Math.min(1, Math.max(0, proportion))
    if (this._bodyProportions === proportion) {
        return
    }
    this._bodyProportions = proportion
    this._setBodyProportionVals()
}

_setBodyProportionVals() {
    this.idealproportionsVal = Math.max(0, this._bodyProportions * 2 - 1)
    this.uncommonproportionsVal = Math.max(0, 1 - this._bodyProportions * 2)

    if (this.idealproportionsVal > this.uncommonproportionsVal) {
        this.regularproportionsVal = 1 - this.idealproportionsVal
    } else { this.regularproportionsVal = 1 - this.uncommonproportionsVal }
}

getBodyProportions() {
    return this._bodyProportions
}

setCaucasian(caucasian, updateModifier = true, sync = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails/Caucasian']
        modifier.setValue(caucasian, false)
        // this.human.targets.applyAll()
        return
    }

    caucasian = _.clamp(caucasian, 0, 1)
    this.caucasianVal = caucasian

    if (sync && !this.blockEthnicUpdates) {
        this._setEthnicVals('caucasian')
    }
}

getCaucasian() {
    return this.caucasianVal
}

setAfrican(african, updateModifier = true, sync = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails/African']
        modifier.setValue(african, false)
        // this.human.targets.applyAll()
        return
    }

    african = _.clamp(african, 0, 1)
    this.africanVal = african

    if (sync && !this.blockEthnicUpdates) {
        this._setEthnicVals('african')
    }
}

getAfrican() {
    return this.africanVal
}
```

```
setAsian(asian, updateModifier = true, sync = true) {
    if (updateModifier) {
        const modifier = this.human.modifiers.children['macrodetails/Asian']
        modifier.setValue(asian, false)
        // this.human.targets.applyAll()
        return null
    }

    this.asianVal = _.clamp(asian, 0, 1)

    if (sync && !this.blockEthnicUpdates) {
        this._setEthnicVals('asian')
    }
    return asian
}

getAsian() {
    return this.asianVal
}

/**
Normalize ethnic values so that they sum to 1.
**/
_setEthnicVals(exclude) {
    const _getVal = ethnic => this[`${ethnic}Val`]

    const _setVal = (ethnic, value) => this[`${ethnic}Val`] = value

    function _closeTo(value, limit, epsilon = 0.001) {
        return Math.abs(value - limit) <= epsilon
    }

    const ethnics = ['african', 'asian', 'caucasian']
    let remaining = 1
    if (exclude) {
        _.pull(ethnics, exclude)
    }
    remaining = 1 - _getVal(exclude)

    const otherTotal = _.sum(ethnics.map(e => _getVal(e)))
    if (otherTotal === 0) {
        // Prevent division by zero
        if (ethnics.length === 3 || _getVal(exclude) === 0) {
        // All values 0, this cannot be. Reset to default values.
            ethnics.forEach((e) => {
                _setVal(e, 1 / 3)
            })
            if (exclude) {
                _setVal(exclude, 1 / 3)
            }
        } else if (exclude && _closeTo(_getVal(exclude), 1)) {
        // One ethnicity is 1, the rest is 0
            ethnics.forEach(e => _setVal(e, 0))
            _setVal(exclude, 1)
        } else {
        // Increase values of other races (that were 0) to hit total sum of 1
            ethnics.forEach(e => _setVal(e, 0.01))
            this._setEthnicVals(exclude) // Re-normalize
        }
    } else {
        ethnics.map(e => _setVal(e, remaining * (_getVal(e) / otherTotal)))
    }
}

/**
Most dominant ethnicity (african, caucasian, asian) or null
**/
getEthnicity() {
    if (this.getAsian() > this.getAfrican() && this.getAsian() > this.getCaucasian()) {
```

```
            return 'asian'
        } else if (this.getAfrican() > this.getAsian() && this.getAfrican() > this.getCaucasian()) {
            return 'african'
        } else if (this.getCaucasian() > this.getAsian() && this.getCaucasian() > this.getAfrican()) {
            return 'caucasian'
        } else {
            return null
        }
    }
}
export default Factors
```

FILE: src/js/human/index.js

Status: MODIFIED | +1 / -0

```
/**
 * @name            MakeHuman
 * @copyright       MakeHuman Team 2001-2016
 * @license         [AGPL3]{@link http://www.makehuman.org/license.php}
+ * @maintainer      Ralf Kruemmel (https://github.com/kruemmel-python/Character-Creator)
 * @description     entry point
 */
export * from './human'
export * from './proxy'
export * from './targets'
export * from './human-modifier'
export * from './ethnic-skin-blender'
export * from './factors'
```